



RESEARCH ARTICLE

10.1029/2022MS003258

Key Points:

- We develop 19 physical and climatological metrics to evaluate 148 subgrid closures online in an open-source idealized ocean model
- Neural network closures perform well online for some filtering operators under these new metrics, but are not robust to distribution shift
- We develop a new hybrid genetic programming and linear regression algorithm and find a symbolic closure with more robust online performance

Correspondence to:

L. Zanna,
laure.zanna@nyu.edu

Citation:

Ross, A., Li, Z., Perezhagin, P., Fernandez-Granda, C., & Zanna, L. (2023). Benchmarking of machine learning ocean subgrid parameterizations in an idealized model. *Journal of Advances in Modeling Earth Systems*, 15, e2022MS003258. <https://doi.org/10.1029/2022MS003258>

Received 17 JUN 2022

Accepted 8 DEC 2022

Author Contributions:

Conceptualization: Andrew Ross, Laure Zanna

Data curation: Andrew Ross

Formal analysis: Andrew Ross, Ziwei Li, Pavel Perezhagin, Laure Zanna

Funding acquisition: Laure Zanna

Investigation: Andrew Ross, Laure Zanna

Methodology: Andrew Ross, Ziwei Li, Pavel Perezhagin, Laure Zanna

Project Administration: Carlos Fernandez-Granda, Laure Zanna

Resources: Laure Zanna

Software: Andrew Ross, Ziwei Li, Pavel Perezhagin, Laure Zanna

Supervision: Andrew Ross, Laure Zanna

Validation: Andrew Ross, Laure Zanna

Visualization: Andrew Ross, Laure Zanna

Writing—original draft: Andrew Ross, Laure Zanna

Writing—review and editing: Andrew Ross, Laure Zanna

Figure preparation: Andrew Ross, Laure Zanna

Table preparation: Andrew Ross, Laure Zanna

Code development: Andrew Ross, Laure Zanna

Model development: Andrew Ross, Laure Zanna

Documentation: Andrew Ross, Laure Zanna

Figure preparation: Andrew Ross, Laure Zanna

Table preparation: Andrew Ross, Laure Zanna

Code development: Andrew Ross, Laure Zanna

Model development: Andrew Ross, Laure Zanna

Documentation: Andrew Ross, Laure Zanna

Figure preparation: Andrew Ross, Laure Zanna

Table preparation: Andrew Ross, Laure Zanna

Code development: Andrew Ross, Laure Zanna

Model development: Andrew Ross, Laure Zanna

Documentation: Andrew Ross, Laure Zanna

Figure preparation: Andrew Ross, Laure Zanna

Table preparation: Andrew Ross, Laure Zanna

Code development: Andrew Ross, Laure Zanna

Model development: Andrew Ross, Laure Zanna

Documentation: Andrew Ross, Laure Zanna

Benchmarking of Machine Learning Ocean Subgrid Parameterizations in an Idealized Model

Andrew Ross¹ , Ziwei Li¹ , Pavel Perezhagin¹, Carlos Fernandez-Granda^{1,2} , and Laure Zanna¹ 

¹Courant Institute of Mathematical Sciences, New York University, New York, NY, USA, ²Center for Data Science, New York University, New York, NY, USA

Abstract Recently, a growing number of studies have used machine learning (ML) models to parameterize computationally intensive subgrid-scale processes in ocean models. Such studies typically train ML models with filtered and coarse-grained high-resolution data and evaluate their predictive performance offline, before implementing them in a coarse resolution model and assessing their online performance. In this work, we systematically benchmark the online performance of such models, their generalization to domains not encountered during training, and their sensitivity to data set design choices. We apply this proposed framework to compare a large number of physical and neural network (NN)-based parameterizations. We find that the choice of filtering and coarse-graining operator is particularly critical and this choice should be guided by the application. We also show that all of our physics-constrained NNs are stable and perform well when implemented online, but generalize poorly to new regimes. To improve generalization and also interpretability, we propose a novel equation-discovery approach combining linear regression and genetic programming with spatial derivatives. We find this approach performs on par with neural networks on the training domain but generalizes better beyond it. We release code and data to reproduce our results and provide the research community with easy-to-use resources to develop and evaluate additional parameterizations.

Plain Language Summary Accurately predicting climate change requires running intensive computer simulations called climate models. Climate models divide the world into grid cells, solving an approximation of continuous equations that model the true dynamics. For accurate predictions, these cells must be small, or equivalently models must be high-resolution. However, even with modern supercomputers, running many high-resolution simulations is prohibitively expensive. One solution is to run climate models at coarser resolution, but include “subgrid parameterizations” to account for physical processes occurring at finer scales and correct bias. Parameterizations are usually developed by analyzing the continuous equations and empirically determining formulae to predict unresolved effects. However, recent studies have applied machine learning (ML) methods to learn parameterizations automatically from limited high-resolution data. This approach has shown promise, but also introduced new challenges with data set preparation, evaluation, interpretability, and implementation. We provide an open-source framework for learning and evaluating parameterizations in a simplified model of the ocean. We use this framework to evaluate numerous ML methods and analyze how best to prepare data sets. We also develop a method of learning equation-based parameterizations which can be more easily interpreted and implemented. Our approach performs comparably to the best ML parameterizations, but generalizes better to oceanic conditions unseen during training.

1. Introduction

Current state-of-the-art climate models solve geophysical fluid equations on horizontal grids of size 25 km and coarser. Models at this resolution are not able to accurately and sufficiently resolve processes with physical length scales smaller than the model grid, for example, convection in the atmosphere and mesoscale eddies in the ocean. Since increases in computational power will likely not enable climate models to resolve these processes before the effects of climate change ensue (Fox-Kemper et al., 2014; Schneider et al., 2017), we must represent subgrid-scale (SGS) processes with closure models, also known as parameterizations. Yet, these SGS models are some of the largest sources of bias and uncertainties in climate simulations: for example, insufficient representations of transient eddies cause biases in modeled currents and sea surface temperature in the ocean (Griffies et al., 2015; Hewitt et al., 2020), and the precipitation pattern is strongly sensitive to the different subgrid cloud closures, thereby causing significant errors in climate projections (Stevens & Bony, 2013). Therefore, developing robust parameterizations remains an important task toward reliable climate projections.

Supervision: Carlos Fernandez-Granda, Laure Zanna

Validation: Andrew Ross, Ziwei Li, Pavel Perezhogin, Laure Zanna

Visualization: Andrew Ross, Laure Zanna

Writing – original draft: Andrew Ross, Ziwei Li, Pavel Perezhogin, Laure Zanna

Writing – review & editing: Andrew Ross, Ziwei Li, Pavel Perezhogin, Carlos Fernandez-Granda, Laure Zanna

In ocean circulation models, stratified turbulent processes are one of the primary targets of SGS closures (Pope, 2000; Vallis, 2017). Classic SGS models are typically designed with specific goals in mind; for example, to dissipate small-scale enstrophy (Smagorinsky, 1963), to reinject energy at larger scales via backscattering (Jansen & Held, 2014; Jansen et al., 2015), or to improve the representation of heat and tracer transport in the ocean interior (Gent & McWilliams, 1990; Gent et al., 1995; Redi, 1982). However, human choices in the design, formulation, and tuning of these SGS models sometimes lead to poor correlation between parameterized SGS forcing and true SGS forcing as diagnosed from high resolution simulations (Khani & Porté-Agel, 2017). This can result in unrealistic large-scale simulations despite recent progress in the representation of resolved processes (Fox-Kemper et al., 2019; Griffies et al., 2009). These shortcomings call for complementary, more systematic and data-driven approaches.

Recently, an increase in high-resolution observations and simulations combined with advances in machine-learning (ML) methods has propelled a surge in the development of data-driven SGS parameterizations in climate models (Beucler et al., 2021; Bolton & Zanna, 2019; Frezat et al., 2022; Guan et al., 2022; Guillaumin & Zanna, 2021; Krasnopolsky et al., 2010; O’Gorman & Dwyer, 2018; Rasp et al., 2018; Subel et al., 2022; Yuval & O’Gorman, 2021; Zanna & Bolton, 2021). Directly learning from data, ML methods automatically extract relevant information from observations and high-resolution simulations to improve coarse-resolution models at a reduced computational cost. Despite their universal approximation properties (Hornik et al., 1989), popular ML models such as neural networks are often opaque to interpretation and can extrapolate poorly to conditions unseen during training (Bolton & Zanna, 2019; O’Gorman & Dwyer, 2018; Recht et al., 2018; Subel et al., 2022).

The performance of data-driven approaches is greatly influenced by choices that must be made in data set preparation. The formulation of the subgrid forcing term, either in terms of tendency or subgrid-scale fluxes, can affect the stability of parameterized models (Yuval et al., 2021). Different filtering schemes also have significant effects on the online performance of subgrid parameterizations (Frezat et al., 2022; Piomelli et al., 1988; Zhou et al., 2019).

There is currently a vast number of possible choices in terms of ML models, training target formulation, and filtering and coarse-graining methods. However, few studies offer a direct and adequate comparison between data-driven ML methods and physical-based parameterizations. Moreover, well-defined quantitative (rather than qualitative) online metrics are lacking. In this paper, we introduce a family of data sets (Sections 2 and 3) and quantitative metrics (Section 4) for learning and evaluating ocean eddy subgrid parameterizations, both offline and online, using a quasi-geostrophic (QG) simulation (data sets and code are available open-source; see Appendix D). Our online metrics quantify to what extent the time-averaged spectral and distributional properties of parameterized simulations match those of ground-truth high-resolution simulations, as well as whether they improve the accuracy of more predictable short-term dynamics. These metrics make it possible to comprehensively compare numerous parameterizations, and the effects of data set design choices on their performance on the physics of the simulations (e.g., spectral properties), climate (e.g., distributional of variables such as PV), and weather (e.g., the evolution of short-term forecast).

In Section 5, we perform such a study for fully convolutional neural network parameterizations, evaluating how offline and online performance change with different designs of inputs (i.e., types of feature variables—velocity or potential vorticity) and outputs (i.e., formulations of subgrid-scale forcing). Even for the best-performing neural networks, we find poor generalization to flow regimes unseen during training, consistent with previous literature (Bolton & Zanna, 2019; Guan et al., 2022; O’Gorman & Dwyer, 2018; Recht et al., 2018; Subel et al., 2021, 2022).

Motivated by these generalization issues, as well as the lack of interpretability of neural networks, there has been increasing interest in the physical sciences community in symbolic regression (also known as equation discovery). In symbolic regression, instead of an opaque model, the final output of training is a transparent equation, which often generalizes better (Champion et al., 2019; Mojgani et al., 2021; Rudy et al., 2017; Zanna & Bolton, 2020; Zhang & Lin, 2018). Many of these studies perform symbolic regression using sparse linear regression on top of a manually constructed basis of terms representing various operations (e.g., derivatives or multiples) of base features. Although powerful for small numbers of terms, this approach quickly becomes prohibitive because the space and time requirements grow exponentially if we consider higher-order operations.

To address these challenges, in Section 6 we introduce a novel algorithm for equation discovery based on genetic programming, an alternative form of symbolic regression that is stochastic but can more efficiently explore higher-order operations (Koza, 1994; Schmidt & Lipson, 2009; Xing et al., 2022). We adapt this algorithm to search over spatial differential operators, and combine it with linear regression and residual-fitting to more efficiently and accurately fit constants. We find that the discovered expression of symbolic parameterization includes features discovered in prior works, is superior to traditional physics-informed turbulence SGS closures, has similar performance to neural networks in both offline and online metrics, and generalizes better to unseen flow regimes than neural networks and baseline physical parameterizations.

2. Numerical Simulations

This section describes the simulations we use to generate our data sets, which are based on pyqg (Abernathey et al., 2022), a Python library that models quasi-geostrophic (QG) systems using pseudo-spectral methods. QG systems are able to capture the generation of ocean mesoscale eddies, the key process we parameterize in this study, and are often used to develop and test physic-based parameterizations (P. S. Berloff, 2005; Porta Mana & Zanna, 2014; Jansen & Held, 2014). In addition, QG systems are a reasonable approximation to the equations of motion in more realistic ocean models in the limit of strong stratification and rotation. Importantly for this study, which tests numerous parameterizations online, they can be simulated much more efficiently than full-fledged ocean models or GCMs.

2.1. Idealized Two-Layer QG Model

We use a two-layer version of the QG model from pyqg. The model's prognostic variable is potential vorticity (PV), denoted as q_1 in the upper and q_2 in the lower layer:

$$q_m = \nabla^2 \psi_m + (-1)^m \frac{f_0^2}{g'H_m} \Delta \psi, \quad m \in \{1, 2\}, \quad (1)$$

where ψ_m is the streamfunction with depth H_m , $\Delta \psi = (\psi_1 - \psi_2)$, and $\nabla = \langle \frac{\partial}{\partial x}, \frac{\partial}{\partial y} \rangle$ is the horizontal gradient operator. Zonal and meridional velocities are obtained from the streamfunction by the relations $u_m = -\partial_y \psi_m$ and $v_m = \partial_x \psi_m$, for each layer with $m \in \{1, 2\}$. We express the horizontal velocity as a single vector $\mathbf{u}_m = \langle u_m, v_m \rangle$. We use the beta-plane approximation, such that the Coriolis acceleration is a linear function of latitude (y) with slope β , such that $f = f_0 + \beta y$, and g' is the reduced gravity.

The prognostic equations, solved in spectral space, are:

$$\frac{\partial \hat{q}_m}{\partial t} = -\hat{J}(\psi_m, q_m) - ik\beta_m \hat{\psi}_m - ikU_m \hat{q}_m + \delta_{m,2} r_{ek} \kappa^2 \hat{\psi}_2 + \widehat{\text{ssd}}, \quad (2)$$

where ∂_t is the Eulerian time derivative, $\widehat{(\cdot)}$ denotes taking the Fourier transform, and $\kappa = \sqrt{k^2 + l^2}$ is the radial wavenumber, where k and l are zonal and meridional wavenumbers, respectively. $J(A, B) = A_x B_y - A_y B_x$ is the horizontal Jacobian. The mean PV gradient in each layer is $\beta_m = \beta + (-1)^{m+1} \frac{f_0^2}{g'H_m} \Delta U$, where $\Delta U = U_1 - U_2$ is a fixed mean zonal velocity shear between the two fluid layers. The Dirac delta function, $\delta_{m,2}$, indicates that the bottom drag with coefficient r_{ek} is only applied to the second and bottom layer. $\hat{q}$ and $\hat{\psi}$ are related to each other via

$$(\mathbf{M} - \kappa^2 \mathbf{I}) \cdot \begin{bmatrix} \hat{\psi}_1 \\ \hat{\psi}_2 \end{bmatrix} = \begin{bmatrix} \hat{q}_1 \\ \hat{q}_2 \end{bmatrix}, \text{ where } \mathbf{M} = \begin{bmatrix} -\frac{f_0^2}{g'H_1} & \frac{f_0^2}{g'H_1} \\ \frac{f_0^2}{g'H_2} & -\frac{f_0^2}{g'H_2} \end{bmatrix}, \quad (3)$$

such that either q or ψ can independently identify the state of the system.

The model is solved pseudospectrally (Fox & Orszag, 1973) through inverting the velocity field and PV to real space, calculating the Jacobian using real-space PV fluxes, and transforming back to spectral space. The scale-selective dissipation (ssd), written as an additive term in Equation 2, is defined as a highly scale selective operator, which attenuates the last 1/3 of the spatial frequencies of the spatial frequencies of all terms on the right-hand side of Equation 2. More precisely, the operator takes the form of an exponential filter, $F_c(\kappa)$, such that

$$F_c(\kappa^*) = \begin{cases} 1, & \kappa^* < \kappa_c \\ e^{-23.6(\kappa^* - \kappa_c)^4}, & \kappa^* \geq \kappa_c \end{cases} \quad (4)$$

where κ^* is the non-dimensional radial wavenumber and $\kappa_c = 0.65\pi$, the cut-off wavenumber. After each time step, $\hat{q}_m(\kappa^*)$ values are multiplied by $F_c(\kappa^*)$. Similar to the 2/3 dealiasing rule (Orszag, 1971), this filtering scheme reduces aliasing errors in the same range of scales, but additionally provides numerical dissipation necessary for stable simulations. The energetic contribution from the ssd term is relatively small (see Figure D11; energy fluxes are an order of magnitude lower than those shown in Figures 2d–2g, and only nonzero over a narrow range of wavenumbers), which is important for simulations of quasi-2D turbulence (Thuburn et al., 2014).

2.2. Model Setup

We configure the model with a doubly periodic square domain with a size of $L = 10^6$ m, a flat topography, and a total depth of $H = H_1 + H_2$, a fixed mean zonal velocity shear, ΔU with $U_2 = 0$. We set a fixed deformation radius r_d , which is the characteristic scale for baroclinic instability and mesoscale turbulence, using $r_d^2 = \frac{g'}{f_0^2} \frac{H_1 H_2}{H}$ (see Table 1 for parameter values).

We select the model's grid size, Δx , in relation to the deformation radius. To resolve mesoscale eddies, one needs to ensure that $r_d/\Delta x$ is greater than 2 (Hallberg, 2013). With $r_d = 15,000$ m, if we choose a 256×256 grid where $\Delta x_{\text{hires}} = L/256 = 3906.25$ m, then $r_d/\Delta x_{\text{hires}} = 3.84$, so mesoscale turbulence should be well-resolved; if instead we choose $\Delta x_{\text{lores}} = L/64 = 15,625$ m such that $r_d/\Delta x_{\text{lores}} = 0.96$, we expect that the simulation is unrealistic with a lack of mesoscale eddies. In such configuration, we would need to find a parameterization that acts at that resolution to replace the missing turbulent physics. We hereby refer simulations with a grid of 256×256 as “Highres,” and simulations with a grid of 64×64 as “Lores.”

All simulations are run with a numerical timestep $\Delta t = 1$ hr.

We consider two distinguishable flow regimes on which generalization properties of parameterizations can be tested: **eddy configuration**, which leads to the formation of isotropically distributed eddies, and **jet configuration**, which leads to the formation of anisotropic jets. These configurations exemplify the two primary scaling regimes of meridional heat transport (Gallet & Ferrari, 2021), and we will test whether parameterizations learned with data from one generalize to the other. Snapshots from each are visualized in Figure 1, and the pyqg parameters used to generate them are given in Table 1.

2.3. Diagnostics

The physical characteristics of QG systems can be qualitatively represented by various diagnostics such as energy and enstrophy spectra, total kinetic energy and enstrophy, and a spectral energy budget (Marques et al., 2022; Yankovsky et al., 2022). Further, we also use these diagnostics quantitatively to define difference and similarity metrics and compare the performance across different SGS models implemented in low resolution simulations in Section 4.

Resolution has a strong impact on these diagnostics. In Figure 2, we show quasi-steady state statistics (spectra, kinetic energy timeseries, probability density function) from simulations run at multiple resolutions (48×48 , 64×64 , 128×128 , and 256×256 grids). The two higher-resolution simulations ($L/\Delta x \geq 128$) show similar behavior, indicating near-convergence of the statistical characteristics over the wavenumber band containing most of the kinetic energy of mesoscale eddies. The two lower-resolution simulations ($L/\Delta x \leq 64$) show significant differences due to insufficiently resolved turbulent features which affect the flow at all scales.

To identify the energy pathway of the flow, we evaluate spectral fluxes of different terms in the two-layer QG system. Let $E(k, l)$ denote the total spectral energy density of the two-layer system, we have

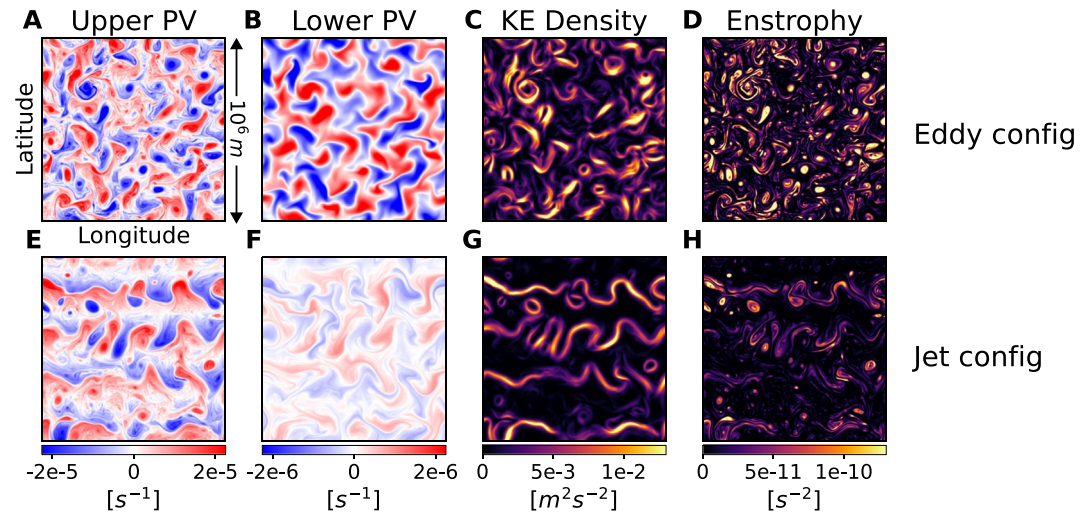


Figure 1. (a and e) Snapshots of upper and (b and f) lower potential vorticity (PV), (c and g) barotropic kinetic energy, and (d and h) barotropic enstrophy for simulations run for 10 years in eddy (a–d) and jet (e–h) configurations over a square, doubly periodic domain of length 10^6 m. Eddy configuration results in an approximately isotropic distribution of vortices, while jet configuration results in the formation of stable, long-lived jets with more coherent latitudinal structure.

$$\begin{aligned} \frac{\partial E(k, l)}{\partial t} = & \sum_{m=1}^2 \frac{H_m}{H} \mathbb{R} [\hat{\psi}_m^* \hat{j}(\psi_m, \nabla^2 \psi_m)] + \frac{f_0^2}{g'H} \mathbb{R} [(\hat{\psi}_1^* - \hat{\psi}_2^*) \hat{j}(\psi_1, \psi_2)] \\ & + \frac{f_0^2}{g'H} k \Delta U \mathbb{R} [j \hat{\psi}_1^* \hat{\psi}_2] - \frac{H_2}{H} r_{ek} \kappa^2 |\hat{\psi}_2|^2, \end{aligned} \quad (5)$$

where $*$ denotes complex conjugate, $\mathbb{R}$ denotes real part, j is the imaginary unit, and the terms on the right-hand side are the spectral contributions from kinetic energy flux (KE flux), available potential energy flux (APE flux), available potential energy generation (APE gen), and bottom drag, respectively.

The lower row of Figure 2 demonstrates the typical energy cycle in QG turbulence (Salmon, 1980; Vallis, 2017): the potential energy of fluctuations is extracted from the prescribed mean flow (APE gen) and cascades toward small scales up to the deformation radius (APE flux) where it is converted to kinetic energy due to baroclinic instability (not shown). The kinetic energy then flows back to large scales following the inverse energy cascade (KE flux), where it is ultimately dissipated by friction (bottom drag).

The coarse resolution models ($L/\Delta x \leq 64$) poorly resolve the formation of mesoscale eddies due to baroclinic instability and their enlargement due to the inverse energy cascade (Zanna et al., 2020), leading to underestimated extraction of energy from the mean flow and a breakdown in the energy cycle.

A promising approach to avoid this breakdown is to supplement the resolved kinetic energy flux with a so-called “backscatter” parameterization (Jansen & Held, 2014; Porta Mana & Zanna, 2014) which energize eddies. We believe that efficient subgrid parameterizations should simulate backscatter at eddy permitting resolution, but also other processes that may matter, such as dissipation. In this paper we do not study precisely which physics are parameterized with data-driven subgrid models, but instead quantify how they influence the resolved energy cycle.

3. Diagnosing Subgrid Forcing

The goal of our work is to learn models that, given only low-resolution inputs, can predict the subgrid forcing, S , missing from a low-resolution QG model (Equation 6). To do that, we need to first quantify subgrid forcing, which is generally done by filtering and coarse-graining high-resolution simulations. This is done sometimes under the implicit assumption that coarsened high-resolution data will have a similar enough distribution to low-resolution data that the same data-driven parameterizations will work for both. We use $(\cdot)$ to denote a generic

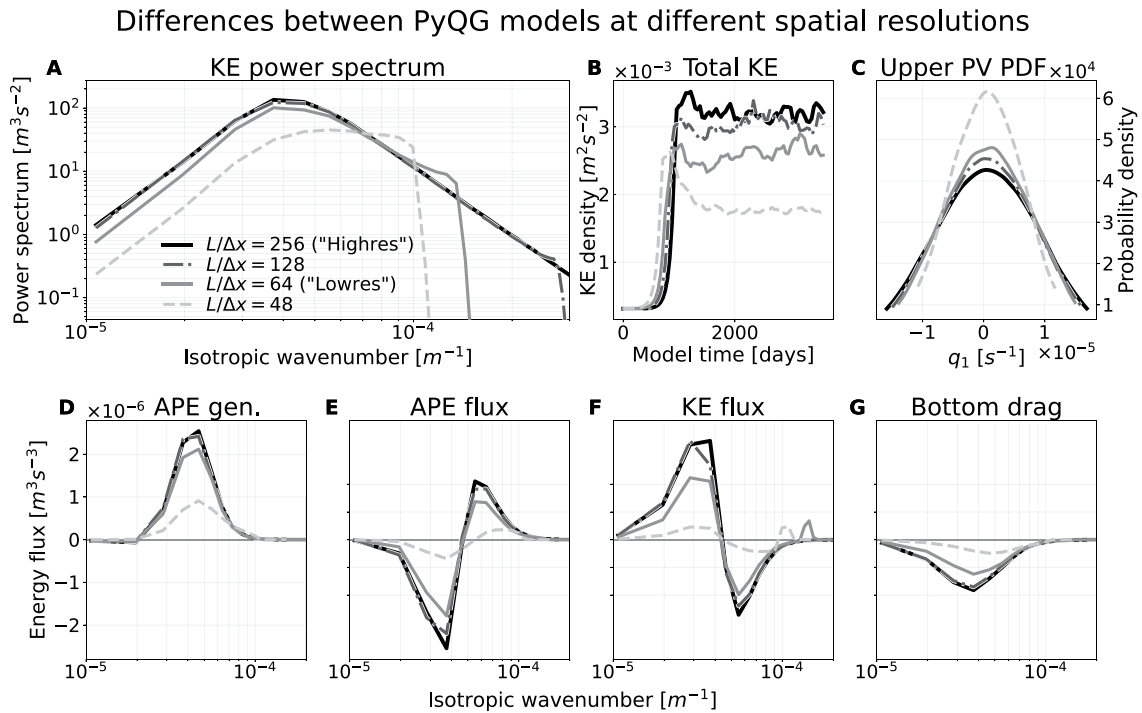


Figure 2. Comparison of time-averaged kinetic energy power spectra summed over fluid layers (a), time-series of total kinetic energy, (b), spatially flattened probability distribution of upper layer PV (c), and spectral energy flux terms (d–g) for eddy configuration simulations at multiple horizontal resolutions: $L/\Delta x = 256$, $L/\Delta x = 128$, $L/\Delta x = 64$, and $L/\Delta x = 48$. Higher-resolution simulations ($L/\Delta x \geq 128$) converge, while lower-resolution simulations ($L/\Delta x \leq 64$) differ from each other and from the higher-resolution simulations.

filtering and coarse-graining operator. However, the choice of filtering and coarse-graining and the choice of subgrid forcing terms to learn are not uniquely defined.

In Sections 3.1 and 3.2, we present several options for how to define subgrid forcing terms in their continuous forms. Specifically, we consider a forcing S in the PV equation that can be added to a coarse resolution simulation to improve its physics such that

$$\frac{\partial \hat{q}_m}{\partial t} = -\hat{J}(\hat{\psi}_m, \hat{q}_m) - ik\beta_m \hat{\psi}_m - ikU_m \hat{q}_m + \delta_{m,2} r_{ek} \kappa^2 \hat{\psi}_2 + \hat{ssd} + \hat{S}, \quad (6)$$

where S can take the form of $\{S_{q_{tot}}, S_q, \bar{\nabla} \cdot \phi_q, \text{curl}(S_u, S_v), \text{curl}(\bar{\nabla} \cdot \Phi_u)\}$, (detailed in Sections 3.1 and 3.2).

In Section 3.3, we discuss the contribution of the forcing term to the energy budget. Finally, in Section 3.4, we describe three different filtering and coarse-graining options applied in this work.

3.1. Subgrid Forcing of Potential Vorticity

We consider three different definitions of subgrid PV forcing for each fluid layer of the QG model: a total tendency, $S_{q_{tot}}$, which is computed online as the residual between the low-res and high-res simulation (e.g., P. S.

Table 1
Table of Parameters Used in Eddy and Jet Configuration

Config.	$\beta \left[\frac{1}{ms} \right]$	$r_{ek} \left[\frac{1}{s} \right]$	H_1 [m]	H_2 [m]	$\Delta U \left[\frac{m}{s} \right]$	$g' \left[\frac{m}{s^2} \right]$	r_d [m]
Eddy	1.5e–11	5.787e–07	500	2,000	0.025	9.81	15,000
Jet	1.0e–11	7.0e–08	500	5,000	0.025	9.81	15,000

Berloff, 2005; Brenowitz & Bretherton, 2018); the subgrid forcing due to nonlinear advection, S_q ; and the subgrid flux divergence forcing, $\bar{\nabla} \cdot \phi_q$.

3.1.1. Total Tendency (Nonlinear Advection and Numerical Dissipation)

Let ∂_t^H and ∂_t^L denote tendency functions from the high- and low-resolution models, respectively (dropping subscripts referring to the model layer for simplicity). For any given high-resolution q , we can express its total subgrid forcing (P. Berloff et al., 2021; Kent et al., 2016; Porta Mana & Zanna, 2014; Shevchenko & Berloff, 2021) due to the differences between the high- and low-resolution models with respect to $(\bar{\cdot})$ as

$$S_{q_{tot}} = \overline{\partial_t^H q} - \partial_t^L \bar{q}. \quad (7)$$

We compute this quantity by setting the initial conditions of the high- and low-resolution models to be q and $\bar{q}$, respectively, taking a single step forward with equal Δt , and subtract the tendency of the low-resolution model from the filtered and coarse-grained tendency of the high-resolution model.

3.1.2. Subgrid Tendency Due To Nonlinear Advection

Another commonly used definition of subgrid forcing considers the unresolved nonlinear advection (Beck et al., 2019; Bolton & Zanna, 2019; Guan et al., 2022; Guillaumin & Zanna, 2021; Maulik et al., 2019; Xie et al., 2020; Zanna & Bolton, 2020), which can be expressed as

$$S_q = \overline{(\mathbf{u} \cdot \nabla) q} - (\bar{\mathbf{u}} \cdot \bar{\nabla}) \bar{q}, \quad (8)$$

where $(\bar{\mathbf{u}} \cdot \bar{\nabla})$ denotes the advection operator defined on the coarse grid. Note that following Grooms et al. (2013) and Porta Mana and Zanna (2014), we define the filtered and coarsened velocity $\bar{\mathbf{u}}$ by inverting the filtered and coarsened PV $\hat{q}$ to $\hat{\psi}$ using Equation 3, multiplying $\hat{\psi}$ by ik and il , and applying an inverse Fast Fourier Transform (FFT) to obtain $\hat{\bar{u}}$ and $\hat{\bar{v}}$, respectively.

3.1.3. Flux Divergence Subgrid Tendency

One difficulty in parameterizing subgrid forcing is that naive ML parameterizations may not obey conservation laws, for example, for momentum and vorticity. Many physical parameterizations are formulated as divergences of fluxes to satisfy conservation laws by the divergence theorem. Ideally, we want to learn ML parameterizations which behave similarly. One approach is to train ML models to predict subgrid forcing (e.g., S_q) but incorporate a numerical divergence operation into their architectures (e.g., as the final layer of a neural network, see Zanna and Bolton [2020]). Another is to diagnose a different quantity whose divergence equals the subgrid forcing (Pawar et al., 2020; Stoffer et al., 2021; Yuval et al., 2021), train ML models to predict this quantity (i.e., the subgrid flux) directly, and compute divergences outside the learned model as part of the implementation of parameterization.

To enable experimentation with this second approach, we define a “subgrid flux” that will be predicted by the FCNN

$$\phi_q = \overline{\mathbf{u} q} - \bar{\mathbf{u}} \bar{q}. \quad (9)$$

Under the assumption that the flow is incompressible (i.e., that $\nabla \cdot \mathbf{u} \approx \bar{\nabla} \cdot \bar{\mathbf{u}} \approx 0$) and that differentiation commutes with filtering and coarsening, we can show that

$$\bar{\nabla} \cdot \phi_q = \bar{\nabla} \cdot (\overline{\mathbf{u} q} - \bar{\mathbf{u}} \bar{q}) \approx S_q.$$

These three formulations (S_q , $S_{q_{tot}}$, and $\bar{\nabla} \cdot \phi_q$) are always highly correlated and often nearly identical, but the exact value of this correlation (especially for $\bar{\nabla} \cdot \phi_q$ vs. the others) can range from 0.75 to $1-10^{-14}$, depending on the layer, timestep, configuration, and especially the filtering and coarse-graining operator (Section 3.4).

3.2. Subgrid Forcing of Velocity

Realistic ocean models use velocity as their prognostic variable with temperature and salinity, rather than PV. Many studies have focused on momentum subgrid closures (Guillaumin & Zanna, 2021; Zanna & Bolton, 2020). Here, we define momentum forcing which is later related to the subgrid PV forcing.

The first definition involves the advection-based subgrid momentum forcing given by

$$\mathbf{S}_u = \overline{(\mathbf{u} \cdot \nabla) \mathbf{u}} - (\bar{\mathbf{u}} \cdot \bar{\nabla}) \bar{\mathbf{u}}. \quad (10)$$

Analogous to Equation 9, we also define momentum subgrid flux terms as

$$\begin{aligned} \phi_u &= \overline{uu} - \bar{u} \bar{u}, \\ \phi_v &= \overline{uv} - \bar{u} \bar{v}. \end{aligned} \quad (11)$$

We use $\Phi_u = (\phi_u, \phi_v)$ to denote the matrix of all four terms of the stress tensor, with a total forcing $\bar{\nabla} \cdot \Phi_u$ where the y -component of ϕ_u is equal to the x -component of ϕ_v .

Given that the PV flux is composed of two parts: the relative vorticity flux and the buoyancy or thickness flux, we note that the relative vorticity flux is related to the momentum flux via the curl operator (Killworth, 1997; Vallis, 2017). In our simulations, we update the PV tendency with the curl of subgrid momentum forcing, for example, $\text{curl}(S_u, S_v) = \partial_x S_v - \partial_y S_u$, which serves as a momentum parameterization in QG equations (similarly for Φ_u). Note that $\text{curl}(S_u, S_v)$ is different from S_q when obtained from the respective coarse-grained fluxes: correlations between the two terms range from +0.2 to −0.4 depending on the filtering and coarse-graining operator.

3.3. Contribution of Forcing to Diagnostics

Similar to Equation 5, we derive the spectral contribution of subgrid-scale forcing toward total energy

$$\left(\frac{\partial E(k, l)}{\partial t} \right)^{\text{sub}} = -\frac{1}{H} \sum_{m=1}^2 H_m \mathbb{R} [\hat{\psi}_m^* \hat{S}_m], \quad (12)$$

where $\hat{S}_m$ denotes the spectral PV tendency induced by the SGS model in the m th layer. This equation states that the total tendency induced by the subgrid term can be written as the projection of subgrid tendency onto the streamfunction in each layer.

3.4. Coarse-Graining and Filtering

We are using a combination of filtering and coarse-graining to diagnose the subgrid forcing. There are a number of possible ways to filter and coarse-grain simulations. Filtering can be identified by various convolutional kernels (top-hat, Gaussian, e.g., Sagaut [2006]), which can be approximated on a given mesh with quadrature rules (Guillaumin & Zanna, 2021; Xie et al., 2020), polynomials based on Laplacian operator (Grooms et al., 2021; Sagaut & Grohens, 1999) or applied in spectral space (Guan et al., 2022). Coarse-graining methods include spectral truncation (Thuburn et al., 2014), averages over boxes (Beck & Kurz, 2021; Porta Mana & Zanna, 2014) or subsampling (Xie et al., 2020, i.e., selection of every K 's point).

The combination of filtering and coarse-graining has also been shown to reduce aliasing in the computation of subgrid forcing (Zanna & Bolton, 2021). Here, rather than focusing on one method for filtering and coarse-graining, we examine the sensitivity of our results to three different operators for diagnosing the subgrid forcing in our simulations: two different filters in spectral space (referred to as “Operator 1” and “Operator 2”), and one filter in real space (and “Operator 3”).

For Operator 1 and Operator 2, given that pyqg is a pseudo-spectral model, it is natural to use spectral methods to perform coarse-graining and filtering. For data generation, we first coarse-grain and then filter, which are commutative for elementwise spectral filtering operators, so can be done in whichever order is most convenient.

Coarse-graining is done by coarse-graining the simulation by a factor of K , or more precisely, truncating the set of spatial modes of $\hat{q}$ by only keeping the first $1/K$. For example, in the case of going from resolution 256×256 to 64×64 , we start with a $\hat{q}$ with 128 modes and only keep the first 32. Spectral filtering generally consists of applying selective decay that reduces the strength of the highest frequencies, whereas low-frequency components are mostly retained after truncation. Here, we use two different filtering methods (Sections 3.4.1 and 3.4.2).

Finally, to mimic the procedure necessary for ocean models which are not run in spectral space, we convert our output to a Cartesian grid and applying filtering and coarse-graining in real space (“Operator 3,” Section 3.4.3).

3.4.1. Operator 1: Spectral Truncation, Sharp Filter

The first option is implemented by simply applying the same quadruple-exponential filter used by pyqg to implement small-scale dissipation in Equation 4. This filter leaves small wavenumbers unchanged but attenuates wavenumbers above a cutoff threshold $\kappa^c \equiv 2/3$ of the low-resolution model's Nyquist frequency:

$$\hat{q}_\kappa = \begin{cases} \hat{q}_\kappa, & \kappa < \kappa^c \\ \hat{q}_\kappa * e^{-23.6(\kappa - \kappa^c)^4 \Delta x_{\text{lores}}^4}, & \kappa \geq \kappa^c. \end{cases} \quad (13)$$

In some sense, this is the most conservative choice of filter possible (i.e., closest to not filtering at all), since it will already be applied within the ocean model. We use “Operator 1” to refer to spectral truncation followed by the application of this filter.

3.4.2. Operator 2: Spectral Truncation, Softer Gaussian Filter

The second spectral filtering option considered (“Operator 2”) is to instead apply the following Gaussian filter to all remaining modes:

$$\hat{q}_\kappa = \hat{q}_\kappa * e^{-\kappa^2 (2\Delta x_{\text{lores}})^2 / 24} \quad (14)$$

This choice of filter is based on Guan et al. (2022) and Pope (2000). According to the definition of the filter width given by Lund (1997), this filter is twice as large as the grid size of the coarse model.

3.4.3. Operator 3: Diffusion-Based Filtering, Real-Space Coarsening

Finally, we consider a procedure which is closer to the procedure needed for ocean models. We apply GCM-Filters (Grooms et al., 2021; Loose et al., 2022), a recent filtering method which approximates the spectral transfer function of Gaussian filter with polynomials based on the Laplacian diffusion operator, converting our pyqg output to a Cartesian grid. We then coarse-grain the filtered output in real space. To reduce the resolution by a factor of K , we average the input field over non-overlapping boxes of $K \times K$ points. We call this procedure “Operator 3.”

A comparison of the effects of these different filtering and coarse-graining operators on PV and its subgrid forcing is shown in Figures 3 and 4.

3.5. Comments Regarding Notation

We use superscripts (1), (2), and (3) (for Operators 1, 2, and 3, respectively) to describe subgrid forcing computed with each operator. For example, $S_q^{(1)}$ signifies the subgrid tendency due to nonlinear advection diagnosed by Equation 8 and computed with the operator from Section 3.4.1, while $\Phi_u^{(3)}$ signifies the tensor of velocity subgrid fluxes diagnosed by Equation 11 and computed with the operator from Section 3.4.3.

In addition, we use $\overline{(\)}$ to refer to all low-resolution variables, whether coarse-grained from a high-resolution simulation or natively from a low-resolution simulation. The reason is that we evaluate parameterizations offline over filtered and coarse-grained high resolution variables, but evaluate parameterizations online over low-resolution variables. Although these variables can be different in various respects (e.g., may be differently distributed), when learning data-driven parameterizations from subgrid forcing data collected offline, we necessarily assume that one will generalize to the other. Though this is an assumption we explicitly test in online evaluation.

Comparing effects of three filtering and coarse-graining operators on potential vorticity forcing

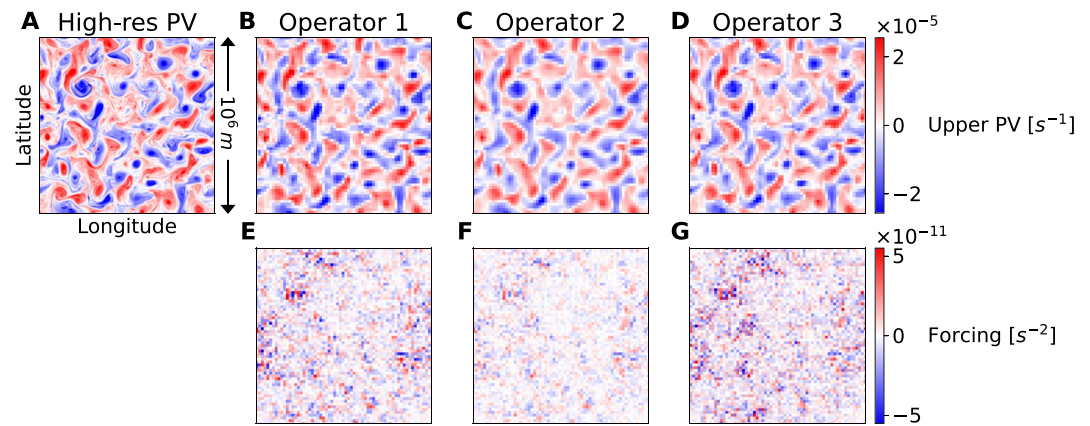


Figure 3. Comparison of the effects of three different methods of filtering and coarse-graining 256×256 eddy configuration initial states (a) to 64×64 (b–d), (e–g) along with resulting forcing terms S_q , defined in Equation 8. (b and e) Operators 1 and (c and f) 2 truncate Fourier modes and apply sharp and soft spectral filters, respectively, while (d and g) Operator 3 applies diffusion-based filtering and averaging in real space. See Section 3.4 for operator definitions and Figure 4 for comparisons of associated spectral properties.

In the remaining text, we also simplify $\bar{\nabla}$ to just ∇ for conciseness. It should be treated as $\bar{\nabla}$ when applied to a coarsened or low-resolution variable.

4. Metrics

In the sections that follow, we evaluate a large number of parameterizations on data generated with different operators and forcing formulations, as well as different inputs and architectures. Given that the models are too numerous to manually inspect, we define several levels of metrics to quantify their performance. In Section 4.1, we define metrics which can be evaluated offline, that is, on held-out testing sets of subgrid forcing data. In Section 4.2, we define online metrics that measure the similarity of low-resolution simulations run with the parameterization to high-resolution simulations. These metrics account for (a) aspects of the model physics (e.g., kinetic energy flux at different scales), (b) the climatological biases and characteristics of key variables (e.g.,

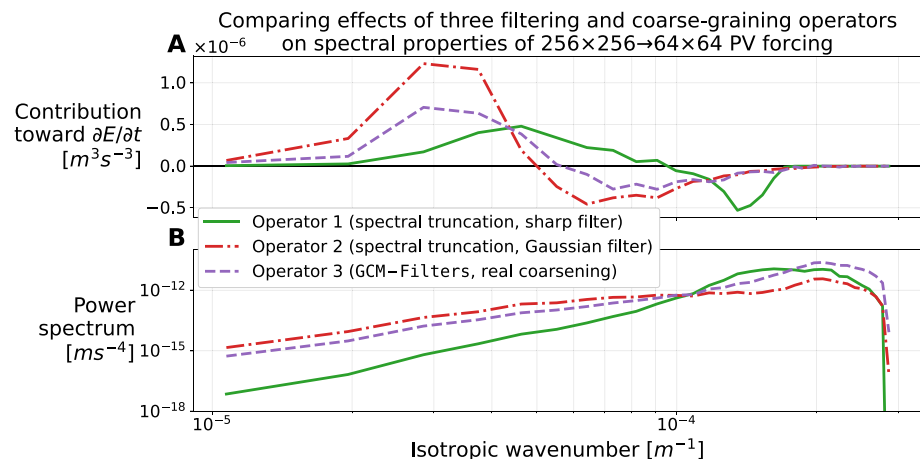


Figure 4. (a) Energy redistribution, Equation 12 and (b) power spectra of S_q by filtering and coarse-graining operator (computed on eddy configuration data, averaged over time, and summed across layers). Each operator produces forcing which redistributes energy differently across scales with different spatial spectra. See Figure 3 for comparisons of forcing snapshots.

distributions of potential vorticity), and (c) the forecast skill of the simulation (e.g., decorrelation timescales of short term forecasts).

4.1. Offline

Offline metrics quantify the parameterization's skill at predicting its intended target. For each fluid layer, we consider:

1. The coefficient of determination (R^2), $1 - \frac{\mathbb{E}[(S-\hat{S})^2]}{\mathbb{E}[(S-\mathbb{E}[S])^2]}$, which is 1 when predictions are perfect, 0 when predictions are no better than always predicting the mean, and negative when worse than always predicting the mean.
2. Pearson correlation (ρ), $\frac{\text{Cov}(S, \hat{S})}{\sigma_S \sigma_{\hat{S}}}$, where σ denotes the empirical standard deviation of a quantity over the data set. This quantity is between -1 and 1 and can remain high even when R^2 is negative, for example, if predictions are wrong by a large but consistent scaling factor.

These metrics are evaluated on held-out data sets of filtered and coarse-grained high-resolution simulations from both eddy and jet configurations. They can either be aggregated over time and space or expressed as functions of time or space. In addition, we visualize the power and energy redistribution spectra of the predicted subgrid forcing and compare them to the corresponding quantities for the ground-truth forcing.

4.2. Online

In contrast to offline metrics, we evaluate online metrics by initializing a new QG simulation at low resolution and, at every time step, passing its state to the parameterization and adding the parameterization's output to the PV tendency. The distribution of these low-resolution states may therefore be different, but by analyzing the ultimate results and testing for various forms of consistency with high-resolution results (i.e., online metrics), we can evaluate whether the parameterization is effective at improving the model physics and/or the climatological or forecast skill.

To compute such online metrics, we first run 5 parameterized low-resolution simulations for 10 years in both eddy and jet configurations initialized from different random states, saving all state variables and diagnostics described in Section 2.3. We then compute distance metrics between the (statistical and spectral) distributions of these variables from the parameterized low-resolution simulation and those from 5 simulations run at high resolution in corresponding configurations. Finally, we normalize these distances by the corresponding metrics for *unparameterized* low-resolution simulations to obtain more interpretable similarity scores.

4.2.1. Differences Between Time-Averaged Power Spectra and Fluxes

Some of the most important characteristics of simulations are how energy and enstrophy distribute and flow across scales, which we measure using power spectra and the spectral flux diagnostics described in Section 2.3. Ideally, a parameterized simulation should match a high-resolution simulation with respect to all such quantities.

For both power spectra and fluxes, we compute a total root mean squared difference between curves f :

$$\text{spectral_diff}(\text{sim1}, \text{sim2}; f) \equiv \sqrt{\frac{1}{|\mathcal{K}|} \sum_{k \in \mathcal{K}} (f_{\text{sim1}}(k) - f_{\text{sim2}}(k))^2} \quad (15)$$

where $\mathcal{K}$ is a suitably chosen set of isotropic wavenumbers common to both simulations. In our case, $\mathcal{K}$ is evenly distributed in log space and is up to 2/3 of the Nyquist frequency of the low-resolution simulation ($\approx 1.07 \times 10^{-5} \text{m}^{-1}$). We compute this metric for the energy and enstrophy power spectra in each layer and for the spectral energy fluxes (KE flux, APE flux, APE generation, and bottom drag), yielding a total of 8 metrics. The contribution of parameterizations toward total energy (Equation 12) is added onto the KE flux term for parameterized low-resolution simulations. An illustration of this kind of distance metric is shown in Figure 5a.

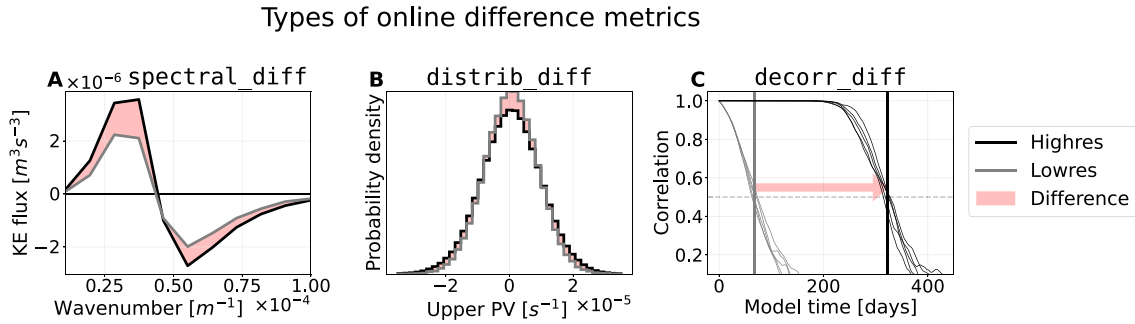


Figure 5. Illustration of the three types of difference metrics defined in Section 4.2. (a) `spectral_diffs` compute the RMSE between different quantities summed over isotropic wavenumber κ . (b) `distrib_diffs` compute the earth mover's distance between the marginal distributions of variables at the end of the simulation. (c) `decorr_diff` estimates how much faster a given simulation diverges from a high-resolution simulation when starting from the same random initial state.

4.2.2. Differences Between Spatially Flattened Probability Distributions

We also consider differences between the empirical distributions of various quantities in different simulations at the end of the simulation, which we measure with earth mover's distance or Wasserstein distance (Monge, 1781; Rubner et al., 2000):

$$\text{distrib_diff}(\text{sim1}, \text{sim2}; f) \equiv \int_{-\infty}^{\infty} |P_{\text{sim1}}(f \leq x) - P_{\text{sim2}}(f \leq x)| dx, \quad (16)$$

where $P_{\text{sim}}(f \leq x)$ is a cumulative distribution function of quantity f in a given simulation. If we imagine the two probability density functions as mounds of earth, this metric corresponds to the minimum amount of work required to move all the mass from one mound to the other. For 1-dimensional distributions, it reduces to the integral of the difference in each cumulative distribution function, which we approximate empirically. We compute these differences for the quasi-steady-state distributions (marginalized over space and at the final timestep) of u , v , q , the kinetic energy density $(u^2 + v^2)/2$, and enstrophy $\text{curl}^2(\mathbf{u})/2$ at each layer. This leads to 10 total metrics for each simulation.

Note that when comparing low-resolution to high-resolution metrics, we are comparing the distributions of, for example, u and $\bar{u}$, so histograms are appropriately normalized. An illustration of this kind of comparison is shown in Figure 5b, though for brevity we show only the difference in the integrals of PDFs rather than the integrals of the corresponding CDFs (which gives the exact value of `distrib_diff`).

4.2.3. Differences in Decorrelation Times

The previous metrics consider whether aggregate, long-term simulation statistics (i.e., “climate”) match those of high-resolution simulations. Arguably, though, parameterizations should also improve the similarity of short-term trajectories (i.e., “weather”) between low- and high-resolution simulations—or at least not significantly worsen it.

We measure this short-term similarity by defining a “decorrelation time” metric, that is, minimum time t to achieve correlation δ from above, averaged over ensemble of initial conditions q_0 and their perturbations ϵ

$$\text{decorr_time}(\text{sim1}, \text{sim2}) \equiv \mathbb{E}_{q_0, \epsilon} \left[\min_t \{ t : \text{Corr}(q_{\text{sim1}}^{(t)}(q_0), q_{\text{sim2}}^{(t)}(q_0 + \epsilon)) \leq \delta \} \right] \quad (17)$$

where each $q_{\text{sim}}^{(t)}(q_0)$ denotes a snapshot of the PV for the given simulation integrated for time t starting from an initial condition q_0 sampled from the quasi-steady state, ϵ is a small independent Gaussian perturbation with standard deviation 10^{-10} , and $\delta = 0.5$. When `sim1` and `sim2` have the same dimensionality, `Corr` denotes the simple Pearson correlation; when they have different dimensionalities (i.e., if `sim1` is higher resolution than `sim2`), we compute the correlation after filtering and coarse-graining the higher-resolution simulation to the resolution of the other simulation using Operator 1 (Equation 13). We approximate expectations $\mathbb{E}_{q_0, \epsilon}$ using empirical averages over 5 random samples of q_0 , ϵ , and we use the same random high-resolution q_0 for all low-resolution models so that correlation trends for different low-resolution models can be paired.

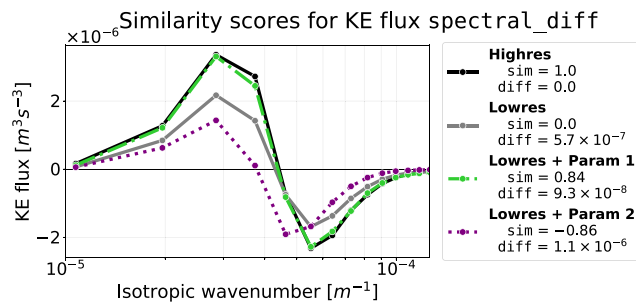


Figure 6. Example online similarity scores for two parameterizations corresponding to the `spectral_diff` of their KE flux terms with respect to high-res (as compared to low-res). In this example, the first parameterized model's KE flux curve is much closer to that of the high-res model than the low-res model, so its similarity is positive and close to 1 (though slightly lower than it might seem from visual inspection due to the logarithmic x-scale). The second parameterized model, on the other hand, is further away than low-res, so it receives a negative score.

4.2.4. From Difference to Similarity

One issue with defining such a variety of distance metrics is that they become difficult to compare especially when they have different units. However, for any particular metric, what we care about is not its actual value but whether it is smaller for parameterized simulations (vis-à-vis Highres simulations) than for low-res simulations. To that end, we re-express our distance metrics as similarity scores that quantify how much closer parameterized models are to high-res than to low-res:

$$\text{Similarity}(\text{param}, \text{high} - \text{res}; \text{diff}) \equiv 1 - \frac{\text{diff}(\text{param}, \text{high} - \text{res})}{\text{diff}(\text{low} - \text{res}, \text{high} - \text{res})}. \quad (19)$$

This similarity score is approximately 1 if the parameterized model's distance to the high-res model is much smaller than that of the low-res model (and exactly 1 for the high-res model); it is approximately 0 if this distance is approximately equal to that of the low-res model (and exactly 0 for the low-res model), and is less than 0 if the distance is larger than that of the low-res model. An example is shown in Figure 6. We also include a validation of the consistency of these scores with respect to high- and low-resolution simulations generated with different random initial conditions in Figure D10. In general, we evaluate our online results using similarity scores.

Note that there are many alternative metrics that could have been selected (e.g., RMSE for decorrelation timescales, Kullback-Leibler for probability distributions (Kullback & Leibler, 1951), absolute error for differences in spectra, etc), that may augment the set defined here to focus on other aspects of the simulations (e.g., extreme events).

4.3. Experimental Setup

With our data sets and metrics now defined, we now describe our experiments to learn and evaluate parameterizations. In total, we test 148 parameterizations—105 fully convolutional neural networks (FCNNs) trained with different data set design decisions (described in Section 5), a hybrid linear and symbolic regression method using genetic programming (described in Section 6), and 42 different parameter settings spread over three baseline physical parameterizations: symbolic regression from Zanna and Bolton (2020), backscatter from Jansen and Held (2014), and Smagorinsky (1963), all three described in Appendix A.

The trained parameterizations are evaluated offline and also implemented into the coarse resolution simulation with 64×64 horizontal resolution for the online evaluation. To simplify the discussion, we begin by describing these categories of parameterizations individually, along with some of the experimental results specific to those categories. We then compare performance across parameterization categories in Section 7.1.

With a decorrelation time now defined, we can compare the expected time it takes for one type of simulation to fall out of sync with another, versus the expected time it takes for one simulation to fall out of sync with a perturbed version of itself:

$$\text{decorr_diff}(\text{sim1}, \text{sim2}) \equiv \text{decorr_time}(\text{sim1}, \text{sim1}) - \text{decorr_time}(\text{sim1}, \text{sim2}), \quad (18)$$

In our study, `sim1` is a high-resolution simulation, which stays correlated with a perturbed version of itself for a relatively long time, while `sim2` will be a low-resolution simulation. With the eddy configuration, the correlation of high-resolution simulations stays above 0.5 for about 1 year (black vertical line in Figure 5c; this roughly quantifies the limit of predictability of the system), whereas unparameterized low-resolution simulations remain >0.5 correlated for about 2 months (gray vertical line in Figure 5c), leading to a `decorr_diff` of 10 months (red arrow in Figure 5c). For a parameterized low-resolution simulation, we might hope that its `decorr_diff` is lower than that of the low-resolution model (e.g., 8 months), indicating that its short-term evolution is more consistent with that of the high-resolution model.

Because we have multiple categories of parameterization (FCNN, genetic programming, and baselines) and multiple categories of online metric (spectral, distributional, and decorrelation time) with numerous individual parameterizations (148) and metrics (19) within each category, we will often simplify as follows. For each category of online metric, we summarize individual parameterization's scores by taking means (or medians and percentiles if visualizing variation). For each category of parameterization, we either show a distribution of these mean scores, or select individual parameterizations to highlight from the Pareto frontier of mean scores within each category, that is, the set of parameterizations which maximize some linear combination of mean scores.

5. Convolutional Neural Network Parameterizations

We consider parameterizations implemented as fully convolutional neural networks (FCNNs) which output predictions for all x, y points simultaneously. Models receive input data at all points x, y in both layers (though we train separate models for each fluid layer to reduce memory cost during training), which allows them to be maximally flexible, and therefore useful for studying the effects of changing attributes of the data set on best-case performance.

5.1. Data Set Design Choices

For our FCNN experiments, we are interested in how the structure of the data set affects the offline and online performance. We train FCNNs to predict subgrid forcing diagnosed with each of the five forcing formulations ($\{S_{q_{tot}}, S_q, \bar{\nabla} \cdot \phi_q, \text{curl}(S_u, S_v), \text{curl}(\bar{\nabla} \cdot \Phi_u)\}$, Section 3), and for each forcing formulation, we generate three FCNNs trained on data sets generated by each filtering and coarse-graining operator (Section 3.4). Finally, we also investigate the effect of the choice of input variables we pass to the FCNN by testing every non-empty element of the power set of $\{\bar{q}, \bar{\mathbf{u}}, \nabla \bar{\mathbf{u}} = (\partial_x \bar{u}, \partial_x \bar{v}, \partial_y \bar{u}, \partial_y \bar{v})\}$, which is 7 options in total. This gives us $5 \times 3 \times 7 = 105$ total options for constructing FCNN parameterizations.

Notation-wise, we refer to models trained on each option as, for example, $\text{FCNN}(\bar{q}, \bar{\mathbf{u}} \rightarrow S_u^{(2)})$, which signifies an FCNN trained on the values of PV and velocity to estimate subgrid momentum forcing (Equation 10), computed with Operator 2 (spectral truncation + Gaussian filter, Section 3.4.2).

For each operator and configuration, we use data from 250 independent high-resolution simulations started from random noise and run for 10 simulation years (generally reaching the quasi-steady state by 3–5 simulation years depending on the configuration; we also include data from the transient spin-up state in the data set). We sample subgrid forcing formulations (i.e., potential prediction targets) and coarsened model state variables (i.e., potential input variables) every 1,000 simulation hours, to remove almost all correlation between successive samples. This gives us six data sets (2 simulation configurations, jet and eddy, $\times 3$ operators) each with 21,750 snapshots of input and target variables (each of which is a $64 \times 64 \times 2$ array).

5.2. Architectural Details and Constraints

Following Guillaumin and Zanna (2021), we train FCNNs with eight fully convolutional layers (128 and 64 filters for the first two layers, respectively and 32 thereafter), ReLU activations, batch normalization after all intermediate layers, and circular padding due to the periodicity of the domain. Each input variable at each fluid layer is passed in a separate input channel. The loss function is mean squared error (MSE), defined as $\mathbb{E}[(S - \hat{S})^2]$, where $\mathbb{E}$ denotes the expected value over a data set and S is a generic prediction target. The FCNNs are trained for 50 epochs on a MSE loss evaluated over minibatches of 64 samples. In preliminary experiments, we found that constraining the FCNNs' final output layers to have zero spatial mean when predicting S_q and S_u was necessary for online numerical stability (as otherwise, q can continually increase, leading to Courant-Friedrichs-Lewy (CFL) condition violations). This is done within the FCNN architecture and not as a post-processing step. The constraint ensures that at each timestep, parameterizations redistribute but not increase or decrease the total PV. However, when predicting ϕ_q and Φ_u , we leave FCNNs unconstrained because we only apply predictions after taking their divergence. Although the chosen architecture could be improved, for example, by adopting the U-Net model of Ronneberger et al. (2015), our goal is not to maximize the performance but to study its relationship with data set design choices.

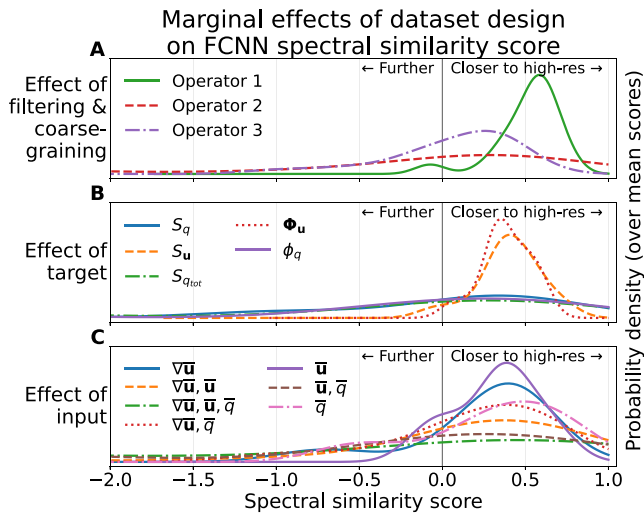


Figure 7. Visualizing the effects of different data set design choices: (a) filtering and coarse-graining operator, (b) forcing formulation, (c) input. We use the probability distributions of mean spectral similarity scores, conditioned on each design choice, and smoothed using kernel density estimation for visual clarity. Similarity score probability mass further to the right (past the 0 line, and toward 1) indicate that the corresponding difference metric was low compared to low-res, therefore indicating good online performance. The results suggest that marginally, similarity was highest along most metrics for parameterizations trained to use velocity (Panel c) to predict velocity-based subgrid forcing (Panel b) calculated with a sharp spectral filter (Panel a).

5.3. Sensitivity of FCNN Performance to Data Set Design

We now present FCNN-specific results of how online performance varies with the data set design choices described in Section 5.1. For each design choice, we constructed the corresponding eddy configuration training data, trained an FCNN parameterization, and evaluated it in both eddy and jet configuration, both offline and online. In each case, all simulations were numerically stable (the CFL condition was not violated). The stability is likely due to our architectural constraints (as discussed above) and perhaps the spectral numerical dissipation scheme of pyqg. However, performance in terms of similarity metrics varied greatly. To visualize this variation, Figure 7 shows the kernel density estimates (Rosenblatt, 1956) of conditional probability distributions of the mean spectral_diff similarity score (substituting Equation 15 into Equation 19) for different data set design choices of filtering and coarse-graining operator, forcing formulation, and input variables. Specifically, these plots show the distribution of the average similarity score across KE power spectra, enstrophy power spectra, and energy budget terms over isotropic wavenumber, conditioned on different choices of filter and coarse-graining (Figure 7a), targeted forcing formulation (Figure 7b), and input variables (Figure 7c). Probability density closer to 1 indicates better performance. Overall, we see higher spectral similarity scores for FCNNs trained on data generated with Operator 1 (spectral truncation with sharp filter) (Figure 7a) and predicting momentum forcing rather than PV forcing (Figure 7b). The choice of input has a weaker impact on these scores (Figure 7c), though simpler terms ($\bar{u}$, $\nabla \bar{u}$, or $\bar{q}$ alone) do slightly better, consistent with (Dresdner et al., 2022). The same results hold for distrib_diff similarity (not shown), which is strongly correlated with spectral_diff (Figure 8). In addition, we can gain insights through

analyzing specific models. If we look at the Pareto frontier of eddy-configuration distributional and spectral similarity across all our experiments (Figure 8), we find that the only Pareto-optimal FCNN predicts $S_{q_{tot}}^{(1)}$, which is computed with Operator 1 but formulated in terms of PV rather than velocity. If we compare this FCNN to others which are identical except for the filtering and coarse-graining operator (Figure D4) or forcing formulation (Figure D5), we find again that the choice of operator continues to matter, but that the forcing formulation has much less effect as FCNNs predicting other forcing formulations with the same filtering and coarse-graining

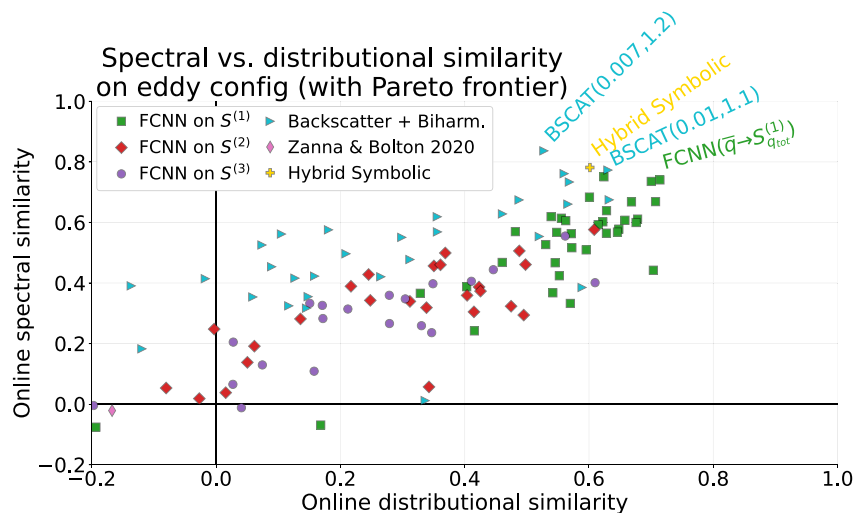


Figure 8. Mean eddy-configuration distributional and spectral similarity scores for many of the 148 parameterizations tested, with those defining the Pareto frontier shown with text (the runs with remaining parameterizations, including all Smagorinsky runs, have scores to the lower-left of the plot range.).

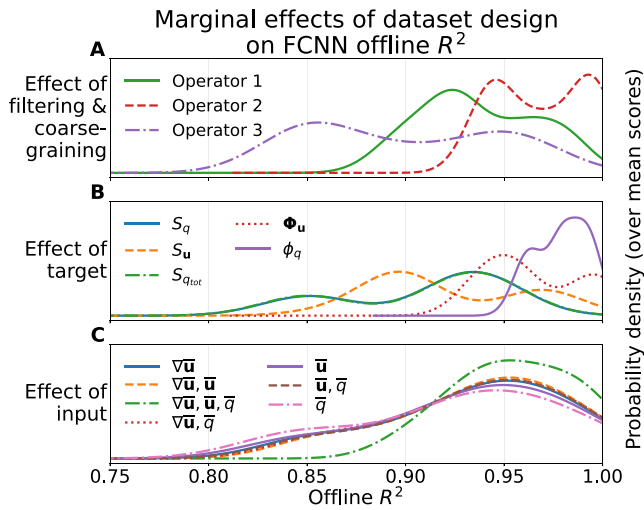


Figure 9. Offline R^2 scores by data set design choice as in Figure 7, almost all of which achieve an R^2 of above 0.8 regardless of condition. The best models by offline R^2 are different from those in Figure 7.

operator all have near-identical effects. Combining these individual results with the aggregate results of Figure 7, our overall interpretation is that (a) the choice of operator is the most important for online performance, and (b) predicting velocity forcing (S_u or Φ_u) rather than PV forcing (S_q , $S_{q_{tot}}$, or ϕ_q) is not necessary for optimal performance, but may be more robust to variations in other suboptimal design choices (e.g., picking Operators 2 or 3). Operator 1 is more faithful to the numerics of the coarse-resolution model that we are using in the online evaluation (this is further supported by the lack of backscatter generated using ZB2020, see conclusions).

5.4. Relationship Between Offline and Online FCNN Metrics

Offline performance, measured using R^2 , is strong for all design choices (see Figure 9), though its relationship with online performance depends on the filtering and coarse-graining operator. For Operator 1, we see positive correlations between offline and online performance (Figures 10a and 10d), meaning that higher R^2 parameterizations generally performed better online. However, for Operators 2 and 3, we see low or negative correlations, meaning that improved offline performance was associated with *worse* rather than better online performance. This result underscores the importance of not focusing too much on improving the offline performance of subgrid parameterizations without first demonstrating that such improvements lead to improvements in physical realism online.

5.5. Varying the Evaluation Target

Some studies measure the online performance of parameterized low-resolution models with respect to the filtered and coarse-grained version of high-resolution data (Beck et al., 2019; Guan et al., 2022; Xie et al., 2020), rather than the high-resolution simulation. Calculating similarity scores for coarse-resolution parameterized models

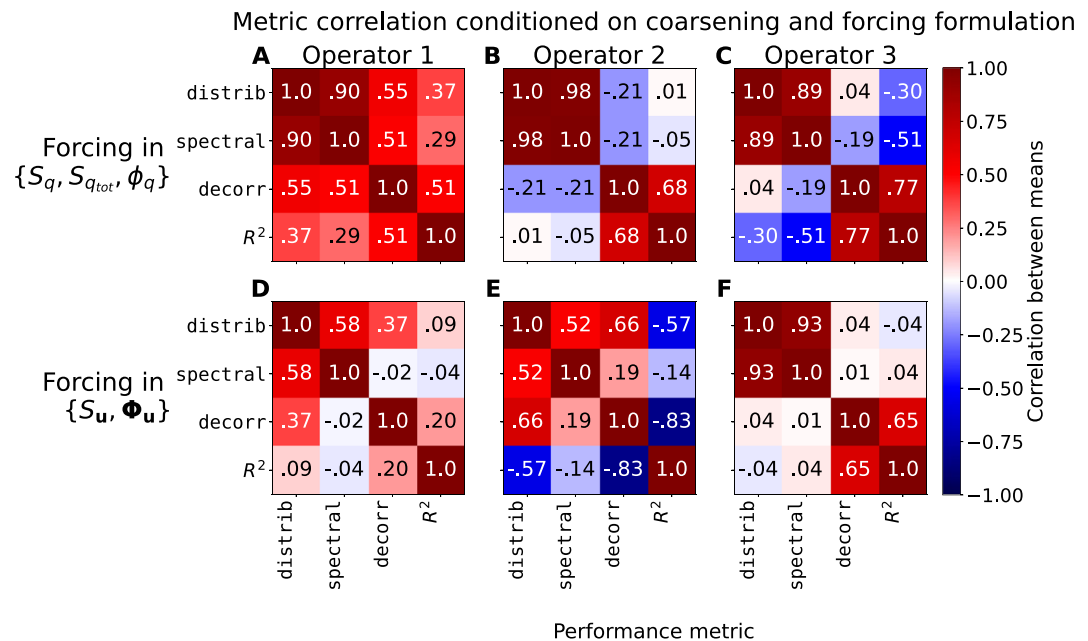


Figure 10. Correlation between FCNNs' mean scores in each metric group conditioned on the filtering and coarse graining operator (columns) and forcing formulation (rows) used to generate their training data. In most cases, distributional and spectral similarity are closely correlated. Correlations with offline R^2 tend to be negative or small, except for FCNNs trained to predict PV forcing variants computed with (a) Operator 1.

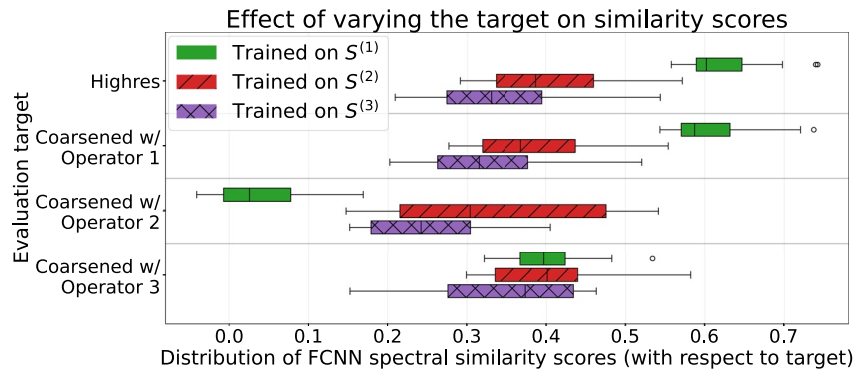


Figure 11. Boxplots showing distribution of `spectral_diff` similarity scores (across all FCNNs trained with each filtering and coarse-graining operator, e.g., $S^{(1)}$ for Operator 1, with any forcing formulation) when the definition of similarity is changed to be relative to a filtered and coarsened version of the high-resolution simulation (bottom three rows), rather than the original high resolution simulation (top row). Center line shows medians, colored bars show the interquartile range (middle 50% of the data), whiskers show positions of nearest points outside twice the interquartile range, and dots show outliers. In general, the relative performance of FCNNs improves when evaluating them against simulations coarsened with the same operator used in their training data. However, absolute performance is only high for FCNNs trained on data from Operator 1 (spectral truncation + maximally sharp filter).

relative to a coarsened and filtered high-resolution simulation increases the scores of top-performing FCNNs, if the parameterization and the target high-resolution models use the same operator (Figure 11). However, even when the target is coarsened with Operator 2 or 3, the actual scores of these models are significantly lower than in the case where the FCNN is trained on data generated by Operator 1. The FCNN trained on data generated by Operator 1 has the best overall spectral similarity score whether we perform the evaluation using the original high-resolution data or data coarsened with Operator 1. This result suggests that Operator 1 is more appropriate for computing subgrid forcing in this data set in an absolute sense.

5.6. Feature Importance for FCNNs

To explore the importance of individual features to our FCNN predictions, we look at snapshots of input gradients, or the partial derivatives of the model's output with respect to its inputs (Baehrens et al., 2010). Note that although there are many proposed methods for quantifying neural network input saliency (Bach et al., 2015; Springenberg et al., 2014), input gradients consistently pass sanity checks that have been developed to validate these methods, while many alternatives do not (Adebayo et al., 2018; Kindermans et al., 2019). In Figure 12,

FCNN($\bar{q} \rightarrow S_{q_{tot}}^{(1)}$) input gradients evaluated at $x = y = L/2$

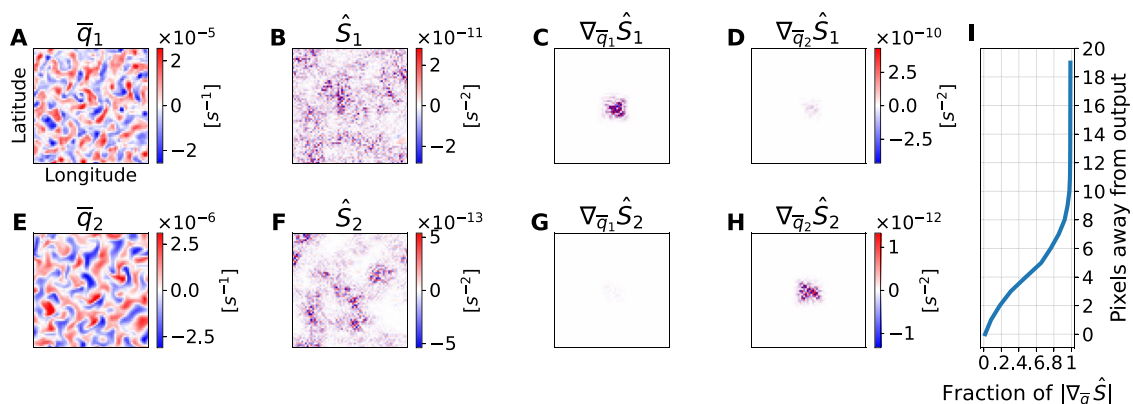


Figure 12. Input gradients of an FCNN mapping $\bar{q}$ (a and e) to $S_{q_{tot}}^{(1)}$ (b and f), evaluated at the center of the domain. Gradient magnitudes are largest around the x , y -position corresponding to the prediction (c and h) in a given layer. However, they still extend relatively far horizontally, needing 9 pixels to reach >95% of their full magnitude (i).

we show a snapshot of input gradients for the FCNN ($\bar{q} \rightarrow S_{tot}^{(1)}$) at the center of the domain, which quantifies the sensitivity of its predictions at this particular location to its inputs. Although our FCNN architecture allows changes in the upper layer $\bar{q}$ to influence the lower layer prediction and vice-versa, this particular FCNN's input gradients are only large in magnitude for $\bar{q}$ in the same layer as the output, suggesting that it largely operates layer-wise. Additionally, gradients were largest in magnitude around the spatial location of the output, suggesting that the model operates locally in the horizontal plane. However, we find that a radius of 5 pixels (fourth-order operations) is needed to explain 50% of the gradients, and a radius of 9 pixels (eighth-order operations) is needed to reach 95% (Figure 12i). This suggests that symbolic parameterizations may need to be fairly non-local and high-order to mimic the behavior of FCNNs. We explore this in the next section.

6. Hybrid Linear and Symbolic Regression and Genetic Programming

In addition to opaque models such as neural networks and random forests, it is also possible to learn equations from data directly with symbolic regression (Koza, 1994). Symbolic-regression based on running sparse linear regression on top of a manually constructed feature library has become popular and achieved impressive results in a number of applications (Brunton et al., 2016; Li et al., 2021). Zanna and Bolton (2020) (ZB2020 hereafter) learned an expression for the subgrid momentum forcing S_u with sparse Bayesian regression (see Equation A7 in Appendix). They used data generated from an idealized primitive equation model, with Gaussian filtering (similar to Operator 2 defined here). Using data from `pyqg` and Operator 2 to calculate the same basis features as in ZB2020 (i.e., divergence, vorticity, stretching and deformation, their x - and y -derivatives, and all cross-multiples), we are able to re-discover Equation A7 with a simple sparse linear regression algorithm. However, sparse linear regression entails trade-offs between the size and expressiveness of the feature library and the complexity and cost of sparse regression, as discussed in Zanna and Bolton (2020). In the example above, our feature library has the initial basis features (4 elements), their first spatial derivatives (8 elements), and all cross-multiples of those initial features (144 elements). If we want to expand this library to consider successively higher-order derivatives (or more than just linear and quadratic multiples), then the number of different expressions we must evaluate for the whole data set will grow exponentially. Additionally, many expressions will be highly correlated, which can prevent many sparse regression algorithms from converging (Hastie et al., 2015).

6.1. Hybrid Genetic Programming (GP)

An alternative approach for symbolic regression is genetic programming (GP), a classic approach in AI (Koza, 1994; Turing, 1950). In contrast to sparse regression, GP algorithms do not require an explicit feature library, simply a set of atomic features and a set of operations for combining them. The GP algorithm then constructs arbitrarily deep expressions by successively applying operators to combine atomic and/or composite features in a randomized fashion, using evolutionary principles to guide a parallel search for an expression that parsimoniously fits the data. More concretely, GP algorithms begin with a “population” of initially short and randomly constructed programs. At each iteration (“generation”), programs are randomly culled, with probability inversely related to their relative performance on a “fitness” metric (see Algorithm 1). Programs that survive can then be randomly modified (“mutated”) in a variety of ways, which can either lengthen or shorten them. This procedure is repeated for a configurable number of generations, after which the GP algorithm returns the best-performing program. To implement genetic programming, we used the `gplearn` Python library (Stephens, 2019). We ran into several difficulties with its default implementation, primarily in its difficulty discovering linear combinations of terms with different orders of magnitude in the weights (constant ranges must be chosen beforehand, and are sampled randomly rather than optimized), as well as the lack of built-in support for spatial differential operators in program evolution. We defined custom `gplearn` functions for differential operators ($\partial/\partial x_i$, ∇^2 , and $\bar{\mathbf{u}} \cdot \nabla$) and combined genetic programming and linear regression in an iterative, residual-fitting procedure described in Algorithm 1. Crucially, in each genetic programming step, we define fitness in terms of correlation rather than absolute error, making fitting the outermost constants unnecessary. We run genetic programming with $\bar{q}$, $\bar{u}$, and $\bar{v}$ as our base features. Arbitrary powers or cross-multiples of these features can be discovered since the operator set includes multiplication. This approach allows us to discover all the same terms which appear in the feature library used for ZB2020, but is not limited to them. Based on results obtained from the FCNNs (Section 5.4), we chose to run our GP method on PV subgrid forcing, S_q , computed with Operator 1 (Section 3.4.1) to simplify learning. Running Algorithm 1 without any manual experimenter intervention leads to a formula of the forcing

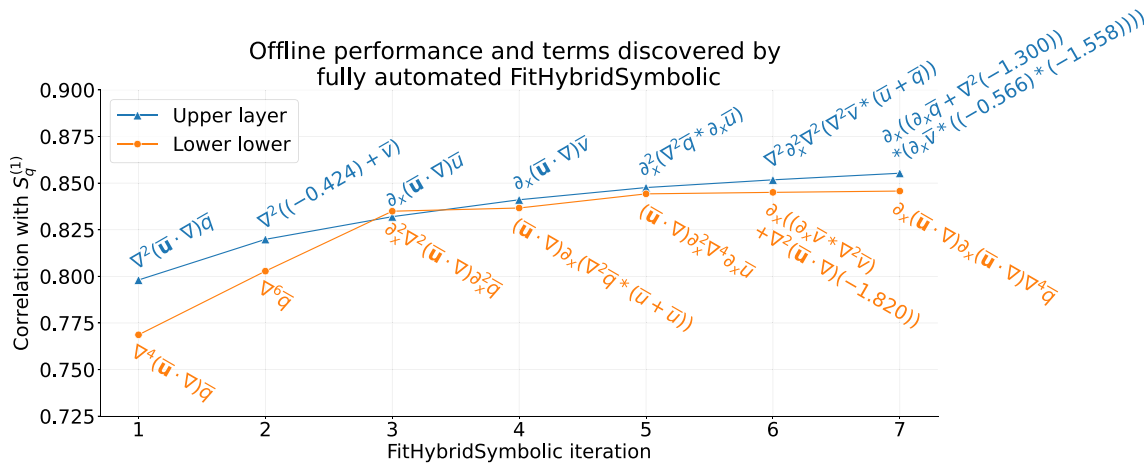


Figure 13. Offline correlation and sequence of terms discovered by—Hybrid Symbolic regression without any human-in-the-loop intervention (terms learned for upper/lower layers in blue/orange respectively). Terms learned in initial iterations tended to be physically meaningful, relatively simple, and related to parameterizations in the literature, while terms learned in later iterations tended to be complex or unphysical (e.g., adding $\bar{u}$ and $\bar{q}$ despite incompatible units in iteration 6).

with an expression for each iteration given in Figure 13. We saw offline performance increase significantly, with many of the discovered features seemingly physically relevant, based on previous published parameterizations. In particular, we note that $\nabla^2(\bar{\mathbf{u}} \cdot \nabla)\bar{q}$ and $\nabla^2\bar{v}$ in the upper layer, together approximate a parameterization proposed by Porta Mana and Zanna (2014), though missing a Eulerian time derivative of PV which is not provided to the algorithm. However, terms discovered *after* the first two iterations tend to vary significantly on random restarts. Terms after the fourth iteration are also significantly harder to interpret (see right end of Figure 13). More importantly, we find that some combinations of terms include additions of terms with different units (e.g., q and u). Finally, implementations of parameterizations using the hybrid expressions found after the sixth iteration were numerically unstable; in addition, although the online performance of runs with the first 4–6 terms from the symbolic parameterizations did improve over low-resolution models, there were still significant differences with respect to many high-resolution diagnostics. To address these issues, we added a human-in-the-loop guidance step described below.

6.2. Human-In-The-Loop Guidance

Some manual intervention can be introduced during the learning procedure to improve interpretability and stability. We added a human-in-the-loop guidance step in each iteration (gray lines in Algorithm 1), where we edited or removed terms that seemed unphysical and sometimes added what seemed like natural extensions of existing terms. In our final OptionalUserEdits step, we attempted to prune the set of terms as much as possible by removing those whose removal did not worsen online performance or adding some that may improve it. We provide an account of our specific actions in Appendix C.

This procedure left us with a final parameterization of the form:

$$S_q^{\text{GP}} = (w_1 \nabla^2 + w_2 \nabla^4 + w_3 \nabla^6)(\bar{\mathbf{u}} \cdot \nabla)\bar{q} + (w_4 \nabla^4 + w_5 \nabla^6)\bar{q} + (\bar{\mathbf{u}} \cdot \nabla)^2 \nabla^2 (w_6 \bar{v}_x + w_7 \bar{u}_y). \quad (20)$$

Here w_i signify the linear weights. Evaluating this parameterization against FCNNs and traditional physics-based models, we find its performance competitive with neural networks in the eddy configuration (Figures 16 and 17) and near-dominant in the jet configuration (18 and 19). We discuss its performance further in Section 7.1 where we compare and contrast different categories of parameterizations.

Algorithm 1. “Hybrid” Linear and Genetic Programming-Based Symbolic Regression (With Optional Human-In-The-Loop Interventions in Light Gray).

```

1: procedure FitGeneticProgram( $x, y$ )
2:   Run gplearn with operators  $\{\partial_x, \partial_y, \nabla^2, (\mathbf{u} \cdot \nabla), *, +\}$ , and
   Fitness(term) =  $|\text{Corr}(\text{term}(x), y)| - 0.001 * \text{Length}(\text{term})$ 
3: end procedure
4:
5: procedure FitLinearRegression( $x, y$ )
6:   Find  $w$  to minimize  $\|w \cdot x - y\|_2^2$ 
7: end procedure
8:
9: procedure FitHybridSymbolic( $x, y$ )
10:  terms  $\leftarrow \emptyset$                                  $\triangleright$  set of symbolic expressions
11:   $w \leftarrow \emptyset$                                  $\triangleright$  weights of those expressions
12:   $\tilde{y} \leftarrow y$                                  $\triangleright$  residual forcing to predict
13:
14:  repeat
15:    for all layers  $z$  do
16:      terms  $\leftarrow$  terms  $\cup$  FitGeneticProgram( $x_z, \tilde{y}_z$ )   $\triangleright$  learn the next term
17:    end for
18:    terms  $\leftarrow$  OptionalUserEdits(terms)
19:    for all layers  $z$  do
20:       $w_z \leftarrow$  FitLinearRegression(terms( $x_z$ ),  $y_z$ )   $\triangleright$  reweight terms
21:       $\tilde{y}_z \leftarrow w_z \cdot \text{terms}(x_z) - y_z$    $\triangleright$  update residuals
22:    end for
23:  until convergence or user decision
24:
25:  return terms,  $w$ 
26: end procedure

```

6.3. Symbolic Regression Feature Importance

As in Section 5.6 for FCNNs, it is useful to quantify the relative importance of the different symbolic terms. One way to do this is by examining the weights w_i . These are visualized in Figure 14 in two ways: (a) as raw values (on a log scale), and (b) normalized after dividing by the standard deviations of the corresponding features (on a linear scale), which makes them directly comparable despite each w_i having different units. In normalized form (Figure 14b), the largest coefficients in both layers are for $\nabla^4(\bar{\mathbf{u}} \cdot \nabla)\bar{q}$, and the absolute magnitudes of these coefficients (Figure 14a) are somewhat close. In contrast, the next-largest normalized coefficients in Figure 14b disagree between layers; for the upper layer, the next-largest coefficient is for $\nabla^6(\bar{\mathbf{u}} \cdot \nabla)\bar{q}$, while the corresponding value in the lower layer is near zero. Instead, the next-largest coefficients in the lower layer are for $\nabla^4 q$ and $\nabla^6 q$, which receive much more weight relative to their magnitudes in the data set. However, despite the difference in relative weight across layers, the absolute magnitudes of the $\nabla^4 q$ and $\nabla^6 q$ coefficients in Figure 14a are almost equal. Overall, these results suggest that the parameterization learns to behave in reasonably similar ways in both layers, but with a few crucial differences, particularly in how they handle $\nabla^6(\bar{\mathbf{u}} \cdot \nabla)\bar{q}$. The final two terms, $(\bar{\mathbf{u}} \cdot \nabla)^2 \nabla^2 \bar{v}_x$ and $(\bar{\mathbf{u}} \cdot \nabla)^2 \nabla^2 \bar{u}_y$, receive relatively little (normalized) weight in either layer. Another way to estimate feature importance is by removing each term, re-fitting the linear regression coefficients, and re-evaluating online performance (Figure 15). If we consider the performance decrease after removal of each feature as a measure of its importance, we reach similar conclusions: the ∇^4 and ∇^6 terms (for both $\bar{q}$ and $(\bar{\mathbf{u}} \cdot \nabla)\bar{q}$) are most important, the $\nabla^2(\bar{\mathbf{u}} \cdot \nabla)\bar{q}$ term is somewhat important, and the $(\bar{\mathbf{u}} \cdot \nabla)^2 \nabla^2$ terms are relatively unimportant.

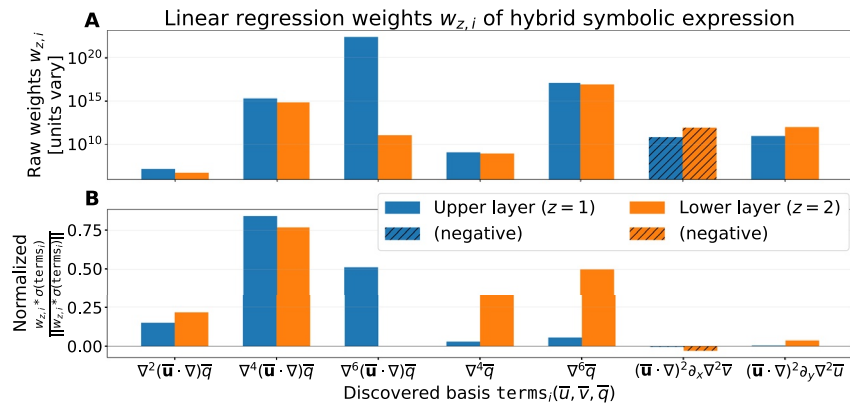


Figure 14. Linear regression-derived weights w for the human-in-the-loop genetic programming-derived basis terms of Equation 20, both as raw values (a, negative values shown with hatching) and normalized (b) after multiplying by the standard deviations of the terms over the training set (giving them consistent units). The absolute magnitudes of many terms are somewhat similar across layers, but their effective contributions to the output differ.

6.4. Interpretation of the Learned Expression

Note that the goal of the paper is not to focus on interpretability but to introduce methods for learning and evaluating parameterizations from data. Therefore, we are not claiming that this parameterization is more physical than anti-viscosity backscatter (Jansen & Held, 2014) or deformation-based parameterizations (Anstey & Zanna, 2017). Nevertheless, we will discuss briefly how the discovered terms compared to other subgrid parameterizations and leave further analysis of their contribution to model physics to future studies. The components of the proposed model were discovered in the following order. In the first few iterations, quadratic expressions, proportional to $(\bar{\mathbf{u}} \cdot \nabla) \bar{q}$, were discovered. Quadratic models are often found to be highly correlated with subgrid forcing (Anstey & Zanna, 2017; Layton & Rebholz, 2012; Meneveau & Katz, 2000; Porta Mana & Zanna, 2014), but often cannot be used as standalone parameterizations. The next few iterations led to eddy-viscosity models, $\nabla^4 \bar{q}$ and $\nabla^6 \bar{q}$. Particularly, both weights w_4 and w_5 being positive implies that there is dissipation of energy in small scales and redistribution to larger scales, that is, backscattering (Jansen & Held, 2014). The final terms discovered are cubic in model variables and contains double-advection operator, $(\bar{\mathbf{u}} \cdot \nabla)^2$. The terms resemble the anticipated PV method from Vallis and Hua (1988). This method allows to preserve properties inherent to geostrophic turbulence such as conservation of energy and dissipation of enstrophy (Marshall & Adcroft, 2010), but it suffers from inaccurate representation of spectral fluxes (Thuburn et al., 2014). In summary, our discovered

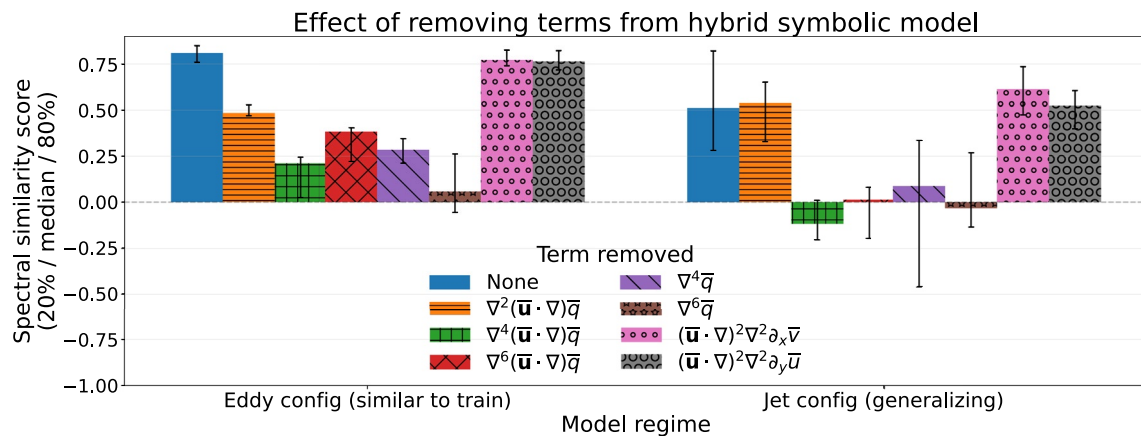


Figure 15. Effect of removing individual terms from the symbolic expression of Equation 20 (using human-in-the-loop guidance) on spectral similarity (median scores within groups, with error bars showing the 20th and 80th percentiles). From left to right, removing $\nabla^2(\bar{\mathbf{u}} \cdot \nabla) \bar{q}$ reduced performance in eddy configuration, but not jet configuration. Removing ∇^4 and ∇^6 terms (for both $\bar{q}$ and $(\bar{\mathbf{u}} \cdot \nabla) \bar{q}$) drastically reduced performance in both configurations, which suggests these terms are crucial. Removing the $(\bar{\mathbf{u}} \cdot \nabla)^2 \nabla^2$ terms had small effects, suggesting they could be dropped for future experiments.

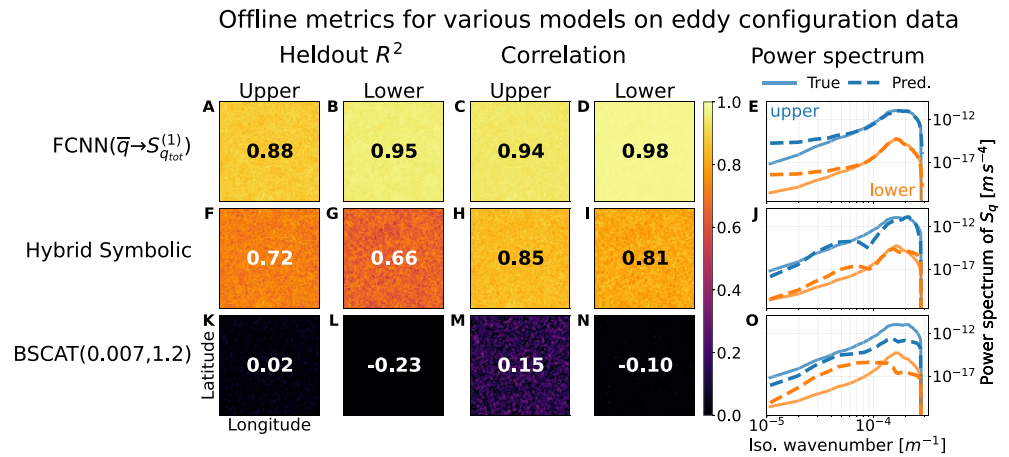


Figure 16. Offline performance for selected subgrid parameterizations on a heldout eddy configuration data set computed with Operator 1, with means shown in spatial plots. FCNN performance (a–e) is strongest overall, though subgrid power spectra diverge slightly at large scales (e). The symbolic regression (f–j) model performs slightly worse, but matches the power spectrum at all scales reasonably well. The backscatter model (k–o) perform much worse offline (though all three perform well online, Figure 17).

closure contains elements of existing subgrid parameterizations, which have pros and cons when used as standalone ones. This symbolic parameterization includes up to the seventh spatial derivative of $\bar{q}$, which may be unrealistic to implement into a climate model. However, it might be more realistic than a fully non-local approach such as the convolutional neural network parameterizations considered in Section 5 or extremely local physics-based parameterizations (such as anti-viscosity).

Selected parameterizations on eddy configuration

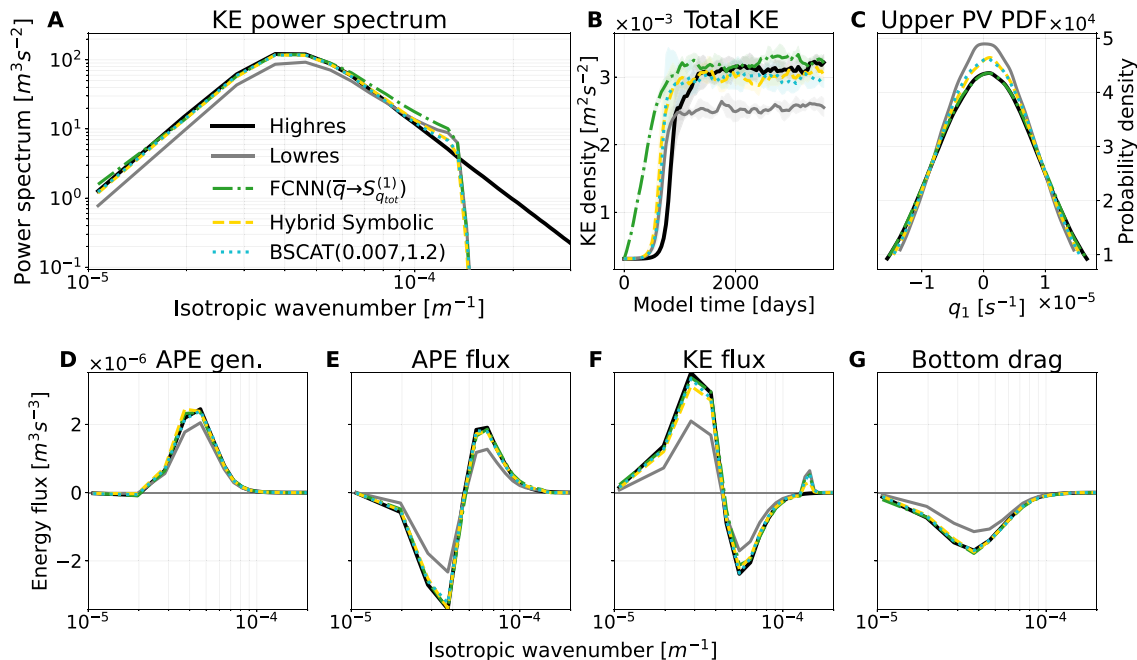


Figure 17. Sample of online performance diagnostics for symbolic regression and best FCNN/backscatter parameterizations by eddy-config `spectral_diff` (taken from the Pareto frontier of top models by spectral and distributional similarity, and averaged across five independent runs). Shading in KE time-series shows standard deviation over runs. All parameterizations improve significantly over the low-resolution model.

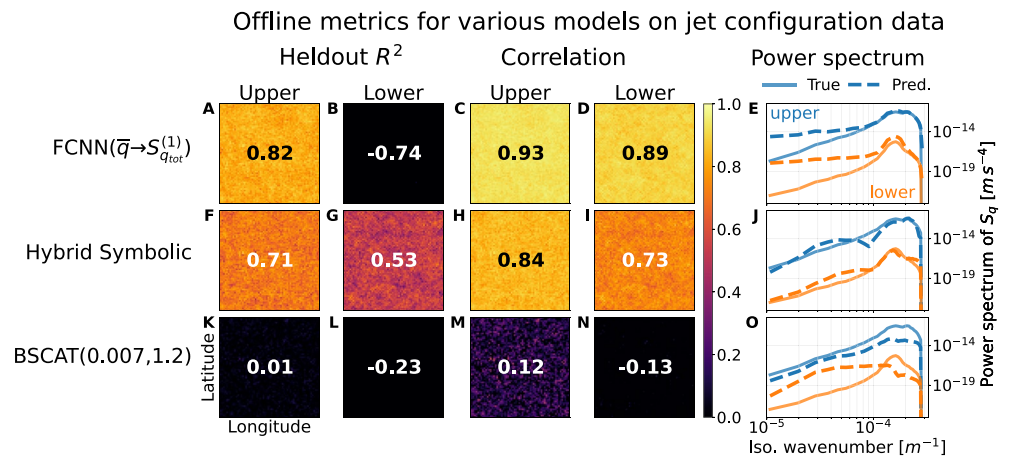


Figure 18. Offline performance as in Figure 18, but testing for generalization to jet configuration. For FCNNs (a–e), R^2 is lower in the upper layer and actually negative in the lower layer. However, correlation remains fairly high, suggesting that performance might improve with rescaling. For our symbolic regression model (f–j) and backscatter (k–o), offline performance remains similar to eddy configuration, though only the hybrid symbolic model generalizes online (Figure 19).

7. Discussion and Conclusion

We will finally compare our top parameterizations and then summarize our key findings in this section.

7.1. Comparing Top Parameterizations

To conclude our analysis, we focus on the top-performing models of different categories. The Pareto frontier of distributional and spectral similarity (Figure 8) conveniently includes one FCNN, our symbolic parameterization, and two backscatter parameterizations (we select the one with higher spectral similarity). Note that the Smagorinsky parameterizations have very poor performance online (not surprisingly since they are dissipative) and we strongly encourage the community to choose better physics baselines when evaluating the performance of data-driven parameterizations. Offline on eddy configuration (Figure 16), FCNN performance (Figures 16a–16e) is strongest overall, though power spectra diverge slightly at large scales (Figure 16e). The symbolic regression model (Figures 16f–16j) performs slightly worse offline than the FCNN, but matches the power spectrum at all scales reasonably well. The backscatter model (Figures 16k–16o) performs much worse offline than the data-driven models, using R^2 as a metric. However, all three selected models perform well online (Figure 17), with the FCNNs showing better distributional performance than the other models (Figure 17c). However, the FCNN models seem to spin up the large scale faster than the other models (Figure 17b). On jet configuration, the offline performance remains similar for all models, except for the R^2 of the FCNN in the lower layer which is significantly lower than for the eddy configuration (Figure 18b). However, online FCNN's performance degrades to significantly worse than the low-resolution without parameterization (Figures 19 and 20). In addition, the backscatter model does not have a significant impact on the low-resolution simulation, though this depends on which metric we consider (e.g., Figure 19). On the other hand, the symbolic model remains fairly robust - without retraining or tuning in this new configuration. FCNNs with different forcing formulations degraded slightly less when transferring to jet configuration (Figure D6). However, their average similarity scores were still low compared to the hybrid symbolic model (Figure D9), and they disrupted the characteristic jet features, causing the flow to more closely resemble the eddy configuration on which they were trained (Figure D3). Even in the eddy configuration, decorr_times for the best-performing models are only modestly closer to those from the high-resolution compared to those of the low-resolution simulation. In the case of the FCNN, the decorrelation times are actually worse (Figure 21) than the low resolution. Using the decorrelation metric, Smagorinsky parameterizations actually performed best (slightly ahead of certain backscatter settings), even though they performed near the worst by all other metrics (see also Figure D8). As expected, the data-driven parameterizations are doing well at representing the averaged statistics at coarse resolution (i.e., the climate) but do not improve the short-term trajectories (i.e., the “weather”).

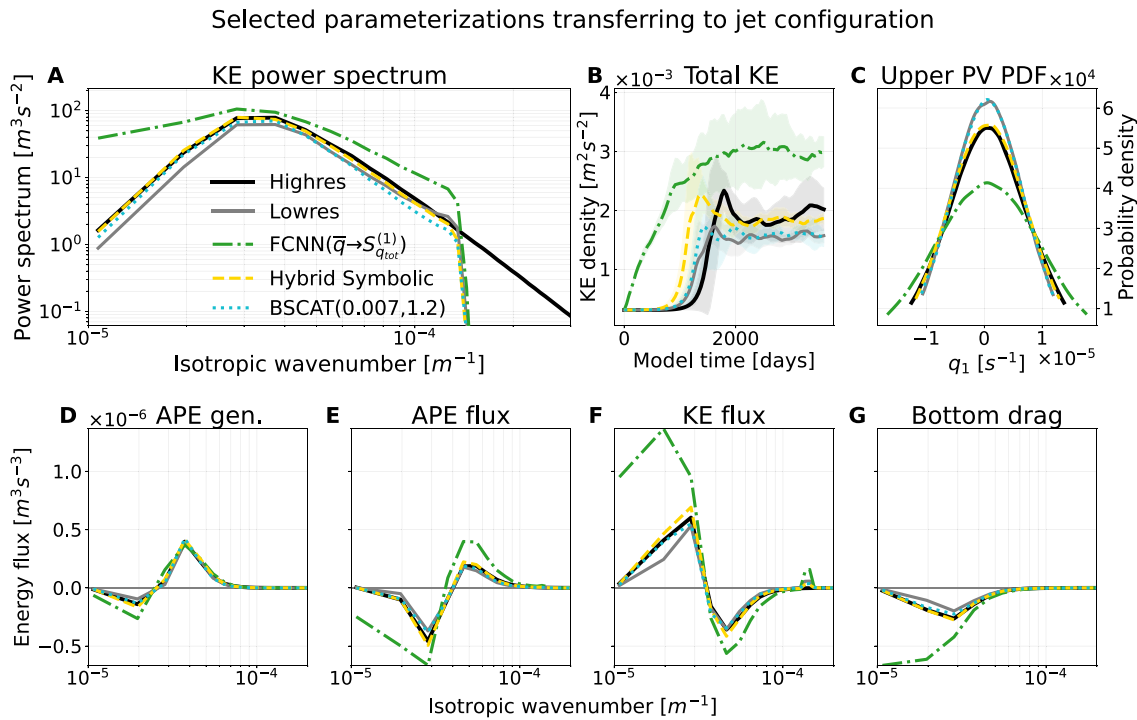


Figure 19. Similar to Figure 17, but evaluated on jet rather than eddy configuration (without retuning). The hybrid symbolic parameterization still improves significantly over low-resolution model, while backscatter has no discernible effect and FCNNs degrade significantly.

7.2. Conclusion

We introduced a framework and a set of data sets for learning and evaluating ocean subgrid forcing parameterizations in a quasi-geostrophic setup, with a focus on a set of well-defined quantitative offline and online metrics. We used this framework to train and test physics-based and data-driven parameterizations under a variety of conditions, namely the different training datasets and definitions of subgrid forcing. Several conclusions stand out as particularly relevant for developing subgrid parameterizations from high-resolution simulations for climate models, even though some of the parameterizations developed here cannot easily be implemented in climate models. We summarize our key points as follows

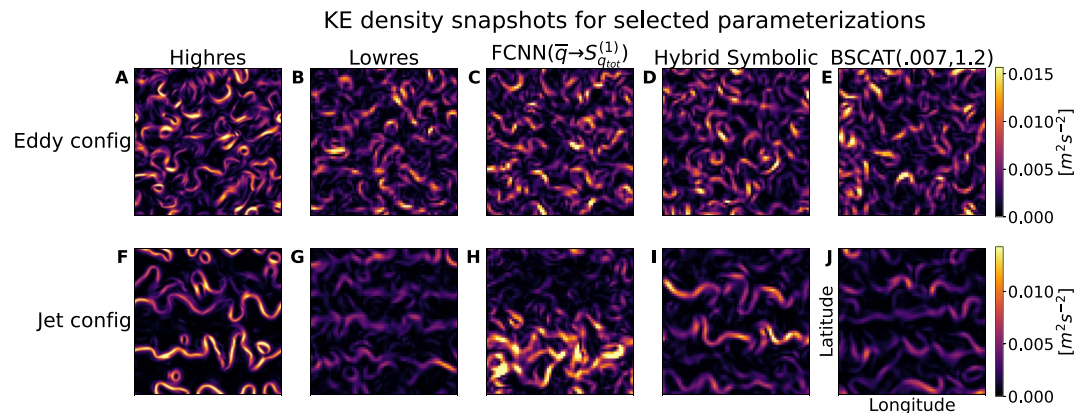


Figure 20. Randomly chosen snapshots of kinetic energy density for selected parameterizations on eddy (a–e) and jet configuration (f–j). On jet configuration, the symbolic parameterization (j) matches high-resolution (f) reasonably well, while the FCNN (i) deviates significantly and backscatter (h) does not appear to have any effect or modify the low-resolution (g). See Figures D1 and D2 for more.

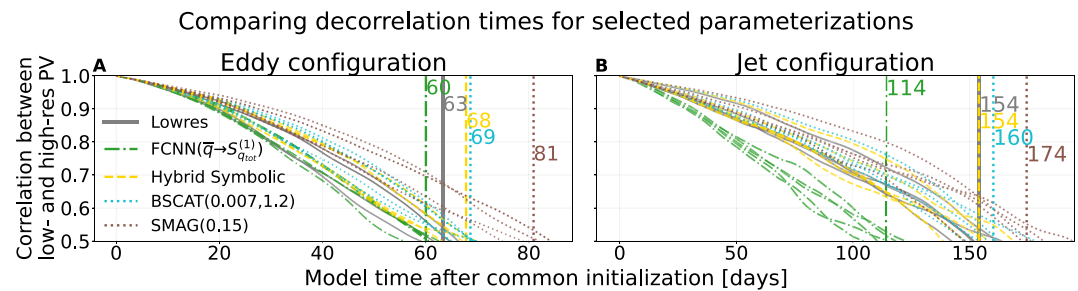


Figure 21. decorr_time results for selected parameterizations on (a) eddy and (b) jet configuration. For each parameterization type, vertical lines show average time for five pairs of high- and low-resolution simulations to reach 0.5 correlation after starting at different randomly sampled initial conditions q_0 . FCNNs diverged from high resolution models faster than unparameterized models, while backscatter and hybrid symbolic parameterizations stayed correlated for similar durations. Smagorinsky parameterizations stayed correlated significantly longer.

- **Metrics:** performance offline and online needs to be rigorously evaluated, rather than eyeballing improvement over a few selected diagnostics, to determine the accuracy and reliability of a given parameterization or simulation. Here, we designed multiple level of metrics: offline metrics that captures the statistics of the subgrid forcing; online metrics that captures the physics of parameterized simulation (e.g., kinetic energy flux) or the climatological and short-term performance of the model (e.g., climatological PDF of potential vorticity, or decorrelation timescales of short term forecasts, respectively). Our open-source framework (Appendix D) will hopefully encourage the research community to find easy-to-use resources for such evaluation and facilitate the development of new parameterizations that more faithfully capture the effects of subgrid-scale processes.
- **Data design choice:** the filtering and coarse-graining operator is key, consistent with Zanna and Bolton (2021) and Frezat et al. (2022). The online results for a given FCNN architecture are highly sensitive to filtering choice; here the best performance was obtained with a filtering that most closely follow the numerics of the model. Therefore, we encourage testing multiple operators for data preparation guided by the target application rather than varying hyperparameters or neural network architectures.
- **Stability:** Our architecturally constrained FCNNs remained numerically stable in any configuration (as shown in Guillaumin and Zanna (2021) for different model configurations), which is likely further aided by the spectral truncation of high-frequency modes in pyqg.
- **Generalization:** symbolic expressions, found using a new algorithm that we developed, were more interpretable with fewer parameters and generalized better to new domains than neural networks, which are infamously sensitive to even minor distributional shifts (Recht et al., 2018).

There are many possible directions we did not explore for NN optimization, including online learning (Dresdner et al., 2022; Frezat et al., 2022; Kochkov et al., 2021; Sirignano et al., 2020; Um et al., 2020), or training on multiple datasets (Bolton & Zanna, 2019; O’Gorman & Dwyer, 2018). New approaches to remain more faithful to the physics of the problem that could be explored as well which include non-dimensionalizing input variables (Beucler et al., 2021), modeling subgrid-scale organization (Shamekh et al., 2022), or finding a better latent space for our input (and eliminating spurious correlation with causal inference). There are also opportunities for improving our symbolic regression procedure, including more intelligently interweaving continuous optimization with genetic programming (Cranmer, 2020), initializing symbolic regression with terms from existing physical parameterizations, or directly learning residuals on top of them. For both neural networks and symbolic regression, finding better metrics for offline learning or testing might help ensure more robust results for online implementation in existing legacy climate models.

Appendix A: Baseline Local Physical Parameterizations

A1. Smagorinsky

A common baseline for physical parameterizations was proposed by Smagorinsky (1963) as scale-selective dissipation. Given the strain-rate tensor, T ,

$$T = \begin{pmatrix} T_{11} & T_{12} \\ T_{21} & T_{22} \end{pmatrix} = \frac{1}{2} \begin{pmatrix} 2\bar{u}_x & \bar{u}_y + \bar{v}_x \\ \bar{u}_y + \bar{v}_x & 2\bar{v}_y \end{pmatrix}, \quad (\text{A1})$$

the Smagorinsky parameterization predicts the subgrid forcing of u and v , denoted as S_{smag} , such that

$$S_{\text{smag}} = \begin{pmatrix} S_{u,\text{smag}} \\ S_{v,\text{smag}} \end{pmatrix} = 2 \begin{pmatrix} (v_{\text{smag}} T_{11})_x + (v_{\text{smag}} T_{12})_y \\ (v_{\text{smag}} T_{21})_x + (v_{\text{smag}} T_{22})_y \end{pmatrix}, \quad (\text{A2})$$

where the short-hands $(\cdot)_{x,y} \equiv \frac{\partial}{\partial x,y}$ are used for low-resolution spatial derivatives,

$$v_{\text{smag}} = (C_S \Delta x)^2 \sqrt{T_{11}^2 + T_{12}^2 + T_{21}^2 + T_{22}^2}, \quad (\text{A3})$$

and C_S is a tunable parameter. Here we will use $C_S \in \{0.075, 0.15, 0.3\}$. Smagorinsky is a parameterization of small-scale dissipation, which can correct the tendency of low-resolution models to concentrate too much energy at small scales. However, the parameterization does not redistribute this energy back up to larger scales via backscatter, as show in theoretical analysis and simulations of quasi-2D turbulence (Kraichnan, 1976; Natale & Cotter, 2017; Thuburn et al., 2014).

A2. Backscatter and Biharmonic Dissipation

Different parameterizations that can potentially address backscatter include the parameterization suggested by Jansen and Held (2014) and Jansen et al. (2015), which consists of scale-selective dissipative operator and an additional negative viscosity part reinjecting energy at larger scales. The magnitude of the negative viscosity part is chosen such that resulting model approximately conserves energy. We adapt this parameterizations for use in pyqg. The small-scale dissipation of enstrophy is parameterized with biharmonic Smagorinsky model (see Equation A3)

$$F_{\text{smag}} = -\nabla^2 [v_{\text{smag}} \nabla^4 \bar{\psi}]. \quad (\text{A4})$$

The negative viscosity backscatter is parameterized with less scale-selective Laplacian viscosity operator:

$$F_{\text{bscat}} = -v_{\text{bscat}} \nabla^4 \bar{\psi}, \quad (\text{A5})$$

and total contribution to PV equation is given as $S_{\text{bscat}} = F_{\text{smag}} + F_{\text{bscat}}$. The negative viscosity backscatter re-injects the C_B fraction of the total energy dissipated by the biharmonic model. As such, the negative viscosity coefficient is given by:

$$v_{\text{bscat}} = C_B \frac{\sum_{i=1}^2 H_i \iint \bar{\psi}_i F_{\text{smag},i} d\bar{x} d\bar{y}}{\sum_{i=1}^2 H_i \iint \bar{\psi}_i \nabla^4 \bar{\psi}_i d\bar{x} d\bar{y}} \quad (\text{A6})$$

where $F_{\text{smag},i}$ is the value of Equation A4 at a particular layer. We run this parameterization at 36 parameter settings corresponding to every combination of $C_B \in \{.7, 0.8, 0.9, 1.0, 1.1, 1.2\}$ and $C_S^2 \in \{.003, .005, .007, .01, .02, .04\}$ (the use of C_S^2 is for convenience).

A3. Zanna Bolton Data-Driven Equation-Discovery Parameterization

Using data from an idealized primitive equation model and relevance vector machine, Zanna and Bolton (2020) learned an expression for the subgrid momentum forcing. They use both barotropic and baroclinic simulated data, and apply Gaussian filtering with coarse-graining to diagnose the subgrid forcing. The form of the parameterization is given by

$$\hat{\mathbf{S}}_{\mathbf{u}}^{\text{ZB2020}} \approx \kappa^{\text{ZB2020}} \nabla \cdot \begin{pmatrix} -\zeta D & \zeta \tilde{D} \\ \zeta \tilde{D} & \zeta D \end{pmatrix} + \mathbf{I} \frac{1}{2} \kappa^{\text{ZB2020}} \nabla (\zeta^2 + D^2 + \tilde{D}^2), \quad (\text{A7})$$

for each vertical layer, with

$$\zeta = \bar{v}_x - \bar{u}_y, \quad \sigma = \bar{u}_x + \bar{v}_y, \quad (\text{A8a})$$

$$D = \bar{u}_y + \bar{v}_x, \quad \tilde{D} = \bar{u}_x - \bar{v}_y, \quad (\text{A8b})$$

where ζ is the relative vorticity, σ is the divergence, and D and $\tilde{D}$ are the shearing and stretching deformation of the low-resolution flow field, respectively. For online tests, rather than using the value of κ^{ZB2020} diagnosed in Zanna and Bolton (2020), we fit the parameter empirically to achieve maximal offline R^2 on the training set (equivalent to that generated using Operator 2). For online simulations, we also test at $\kappa^{\text{ZB2020}} = 2$ and 1/2 times the empirically fit value.

Appendix B: Decomposition of Subgrid Forcing

We can further decompose subgrid contribution into the contribution toward kinetic energy and the contribution toward potential energy. Let S_ψ be the tendency in the streamfunction induced by subgrid forcing, we use Equation 3 to rewrite Equation 12 as

$$\begin{aligned} \left(\frac{\partial E(k, l)}{\partial t} \right)^{\text{sub}} &= -\frac{1}{H} \sum_{m=1}^2 H_m \mathbb{R} \left[\hat{\psi}_m^* \left[(-\kappa^2 \mathbf{I} + \mathbf{M}) \hat{\mathbf{S}}_\psi \right] \right] \\ &= \frac{1}{H} \kappa^2 \sum_{m=1}^2 H_m \mathbb{R} \left[\hat{\psi}_m^* \left(\mathbf{A}_\kappa \hat{\mathbf{S}}_q \right)_m \right] - \frac{1}{H} \sum_{m=1}^2 H_m \mathbb{R} \left[\hat{\psi}_m^* \left(\mathbf{M} \mathbf{A}_\kappa \hat{\mathbf{S}}_q \right)_m \right], \end{aligned} \quad (\text{B1})$$

where $\mathbf{A}_\kappa = (-\kappa^2 \mathbf{I} + \mathbf{M})^{-1}$. On the right-hand side of Equation B1, the first term matches the definition of the contribution toward kinetic energy, and we regard the second term as the contribution toward potential energy. This decomposition is used in calculating the spectral similarity scores.

Appendix C: Human-In-The-Loop Symbolic Regression Steps

In this section, we describe the specific “OptionalUserEdits” steps we took in applying Algorithm 1 to obtain Equation 20. In the first `gplearn` step, we discovered $\nabla^2(\bar{\mathbf{u}} \cdot \nabla) \bar{q}$ (in the upper layer) and $\nabla^4(\bar{\mathbf{u}} \cdot \nabla) \bar{q}$ (in the lower layer), which gave us training set correlations of 0.80 (upper) and 0.77 (lower) after fitting models with both terms to each layer. To this, we added $\nabla^6(\bar{\mathbf{u}} \cdot \nabla) \bar{q}$ to extend the pattern, which brought the same correlations to 0.84 and 0.82. We then ran the next `gplearn` step, which outputted $\nabla^4 \bar{q}$ (upper) and $\nabla^6 \bar{q}$ (lower). This brought correlations up to 0.845 (upper) and 0.836 (lower). We kept both these terms, and experimented with adding $\nabla^8 \bar{q}$, but correlations actually decreased in the lower layer. We then ran the next `gplearn` step, which returned $(\bar{\mathbf{u}} \cdot \nabla)^2 \nabla^2 \partial_x \bar{v}$ and $\partial_x \nabla^8 \bar{q}$. This nudged correlations to 0.846 (upper) and 0.838 (lower), which nudged very slightly higher to 0.846 and 0.840 when further adding the counterparts of these terms obtained by switching x and y , $(\bar{\mathbf{u}} \cdot \nabla)^2 \nabla^2 \partial_y \bar{u}$ and $\partial_y \nabla^8 \bar{q}$. From this set of terms (which includes all terms in Equation 20 with the addition of two ninth-order $\partial_i \nabla^8 \bar{q}$ terms), we began a final OptionalUserEdits step using online performance as a guide (removing each term individually, but pairing up the removals of the terms with natural x and y counterparts). In this step, we found that the $\partial_i \nabla^8 \bar{q}$ terms were actually hampering online performance (i.e., performance rose without them), while the others all appeared to help (i.e., performance fell without them)—though our results in Figure 15 later showed that the slight improvement we saw from the $(\bar{\mathbf{u}} \cdot \nabla)^2 \nabla^2 \partial_i \bar{u}$ terms was not significant. We then accepted the expression of Equation 20 as our final output, saving its weights (learned with respect to eddy-config $\mathcal{S}_q^{(1)}$). Note that because the genetic programming steps are stochastic, re-running this procedure with a different random seed might produce different results. For example, in Figure 13, we discovered a $\nabla^2 \bar{v}$ term in the second step, but in this case such a term was never learned (though this could be alternately explained by the manual addition of $\nabla^6(\bar{\mathbf{u}} \cdot \nabla) \bar{q}$, which may have accounted for its contribution).

Appendix D: Supplementary Figures

This section includes additional result figures (Figures D1, D2, D3, D4, D5, D6, D7, D8, D9, D10, D11, and D12; Tables D1 and D2).

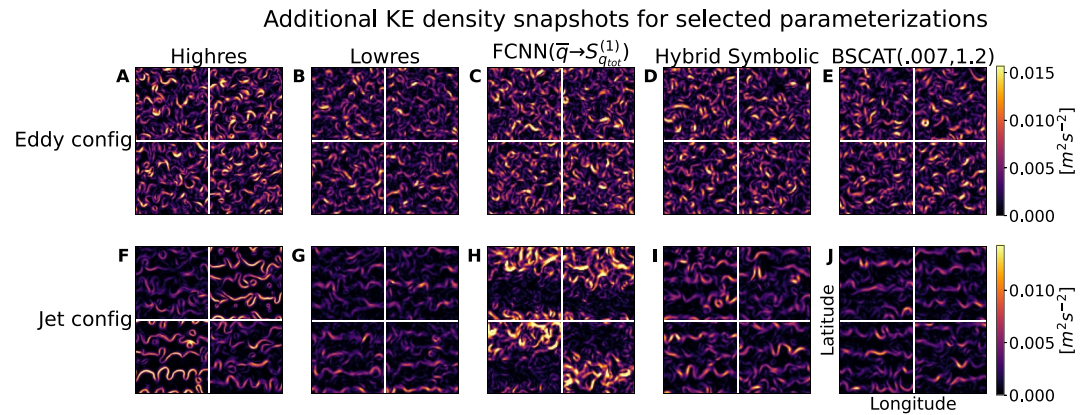


Figure D1. Like Figure 20, but showing additional randomly chosen snapshots.

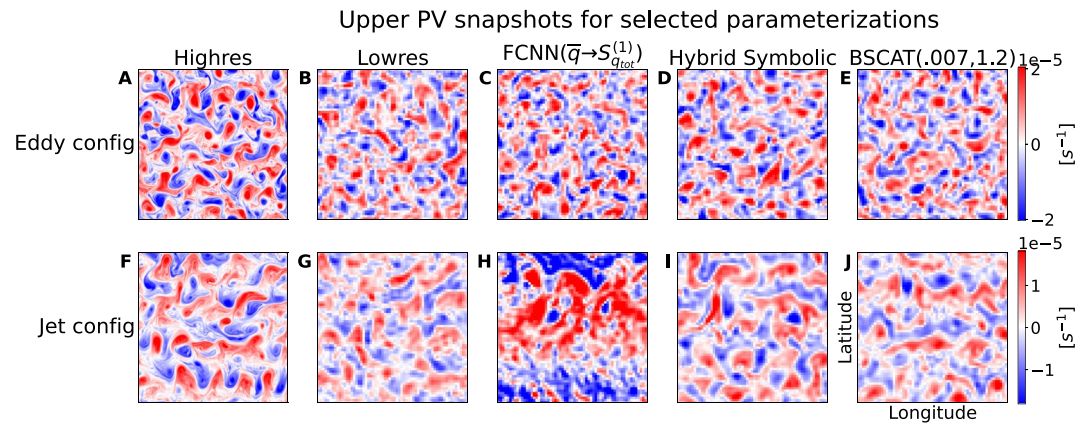


Figure D2. Like Figure 20, but showing upper PV q_1 rather than KE density.

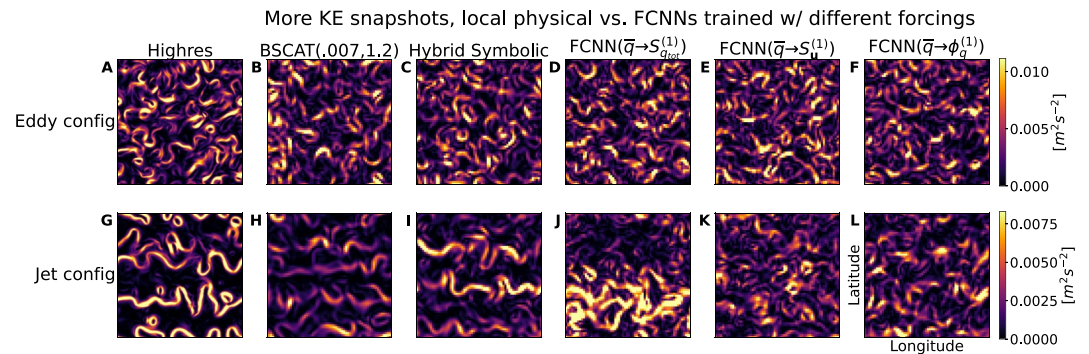


Figure D3. Like Figure 20, but additionally showing KE snapshots for FCNNs trained on eddy configuration data with different forcing formulations (see Figures D5 and D6). All FCNNs produce reasonable results on eddy configuration (d–f), but on jet configuration (j–l), the snapshots do not resemble high-resolution (g), with either latitude-specific increases in energy (j) or disruption of jets in favor of isotropic eddies (k–l), resembling FCNN training conditions.

Comparing FCNNs trained on different operators on eddy configuration

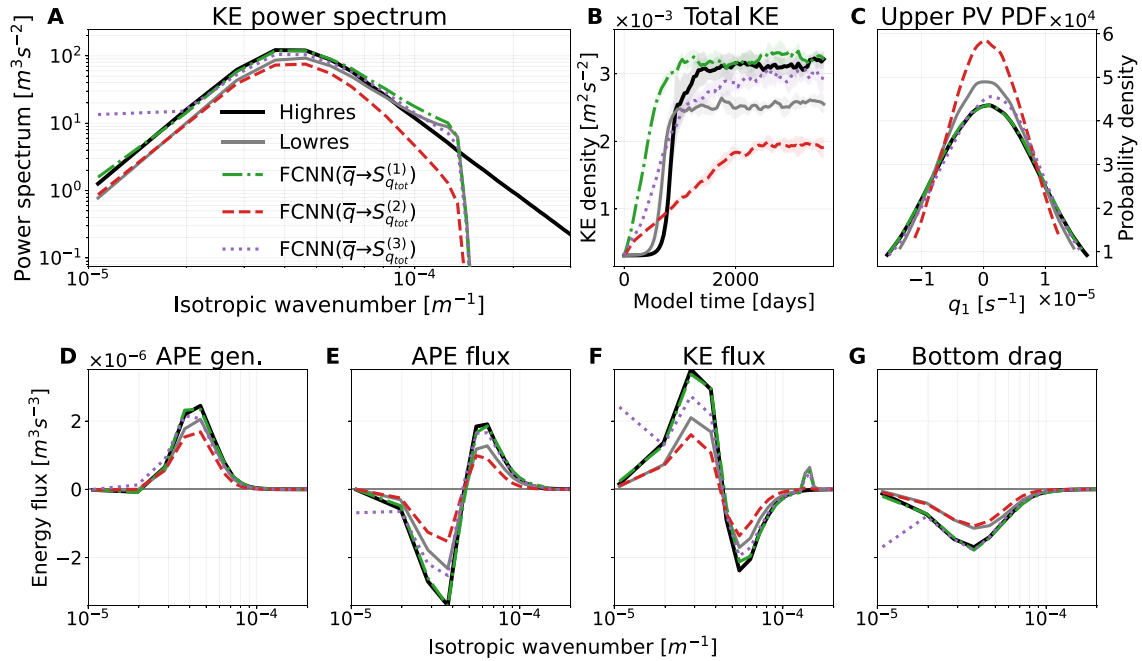


Figure D4. Like Figure 17, but comparing FCNNs trained to predict $S_{q_{tot}}$ computed with each filtering and coarse-graining operator. Only the model trained with Operator 1 (Section 3.4.1) performs near-optimally, though the model trained with Operator 3 (Section 3.4.3) does well except for deviations in spectral metrics at large scales (a, f, and g). These results suggest the filtering and coarse-graining operator is important for parameterization performance.

Comparing FCNNs predicting different forcing formulations on eddy configuration

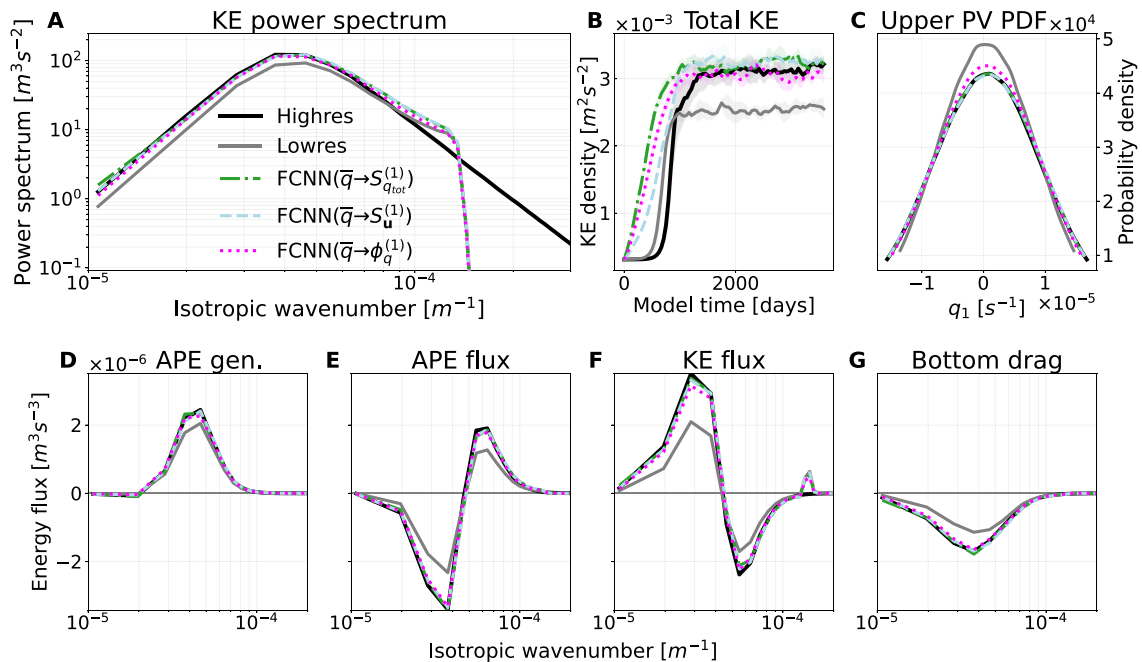


Figure D5. Like Figure 17, but comparing the online eddy configuration performance of FCNNs trained to predict different subgrid forcing formulations ($S_{q_{tot}}$, S_u , and ϕ_q) computed with Operator 1 (Equation 13). All perform almost equally well, suggesting that the forcing formulation may matter much less than the filtering and coarse-graining operator (Figure D4).

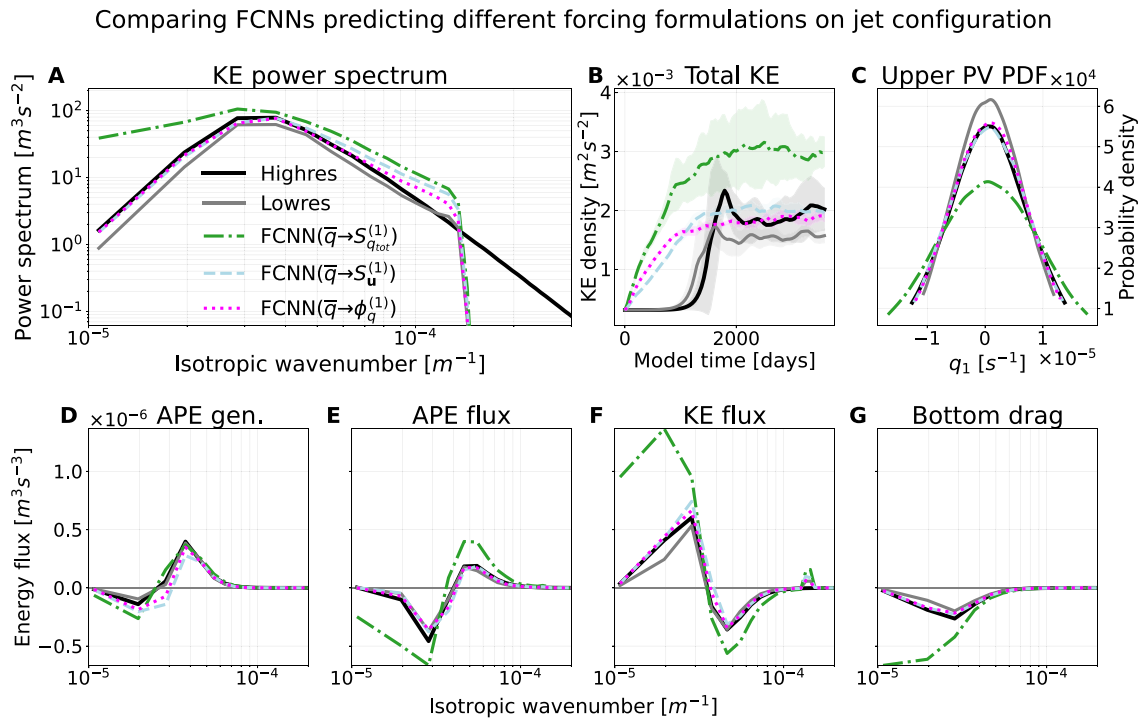


Figure D6. Like Figure 19, but comparing the online jet configuration performance of FCNNs trained to predict different subgrid forcing formulations ($S_{q_{tot}}$, S_u , and ϕ_q) computed with Operator 1 (Equation 13). In this case, the models trained to predict $S_u^{(1)}$ and $\phi_q^{(1)}$ appear to generalize better. However, their average scores across the full set of metrics (e.g., Figure D9) remain low, and in KE snapshots from these FCNNs (Figure D3), the characteristic jet behavior we see in high-resolution is absent.

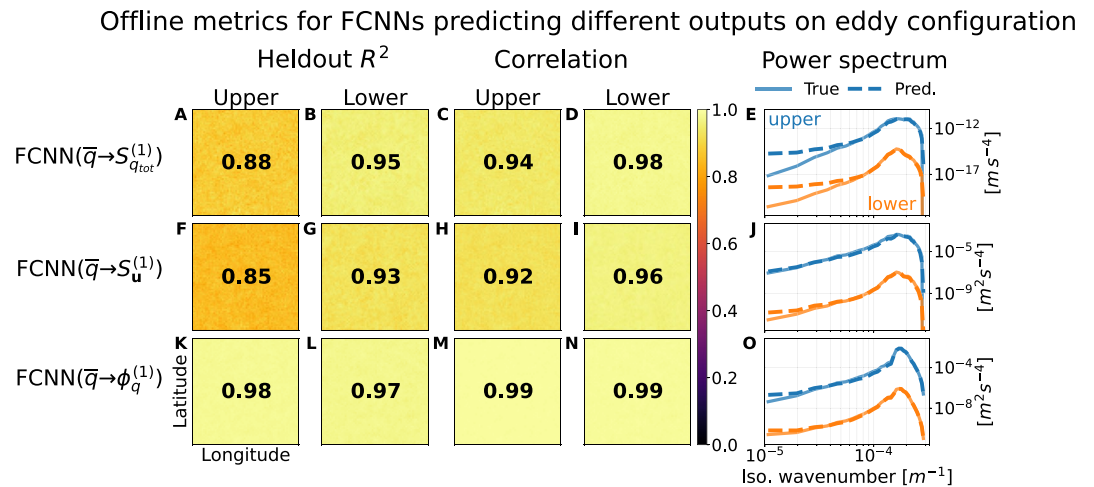


Figure D7. Offline results for more forcing formulations ($S_u^{(1)}$ and $\phi_q^{(1)}$ results show averages over u and v terms). Many performance metrics are generally higher for models trained to predict subgrid fluxes (k–n), but this difference disappears if we compute them with respect to the implied subgrid forcing (i.e., by taking the divergence of the predicted quantities and comparing that to the true subgrid forcing, rather than comparing predicted to true subgrid fluxes).

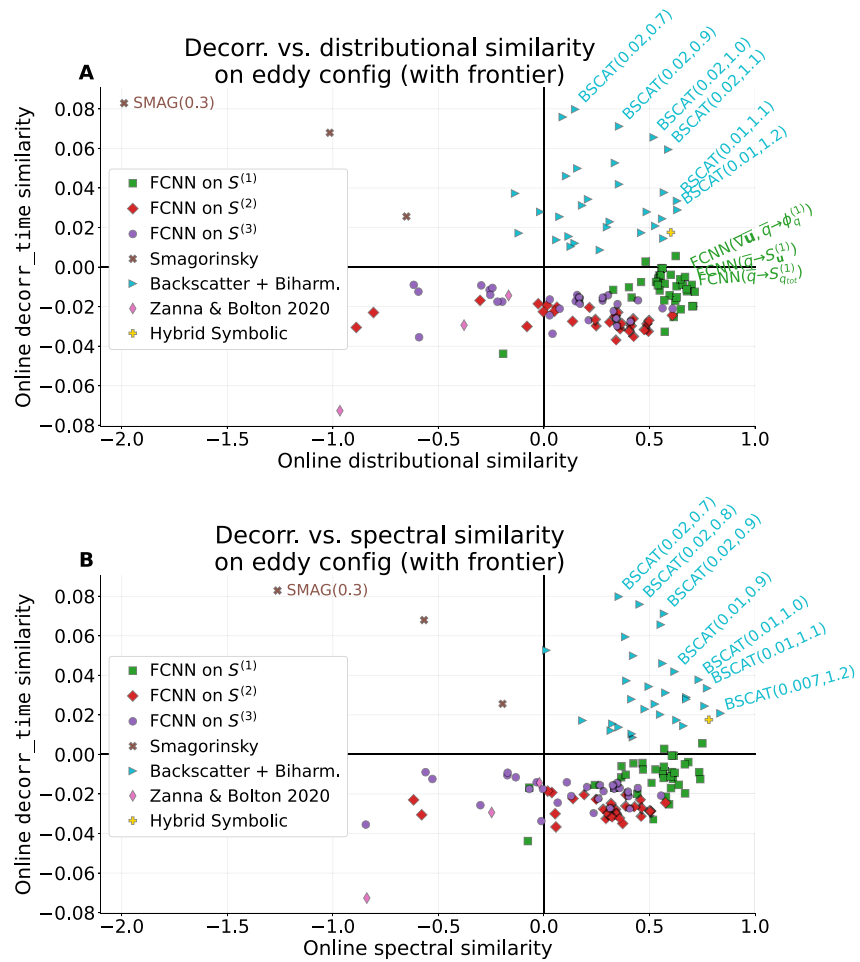


Figure D8. Like Figure 8, but comparing distributional similarity from Section 4.2.2 (a) and spectral similarity from Section 4.2.1 (b) with decorrelation time similarity from Section 4.2.3. Smagorinsky and backscatter parameterizations (which form most of the Pareto frontier in both plots) increase decorrelation time, though only by about 8% of the gap between low- and high-resolution decorrelation times (which is what the y-axis signifies). Neural networks almost universally reduce it, while the hybrid symbolic parameterization modestly increases it.

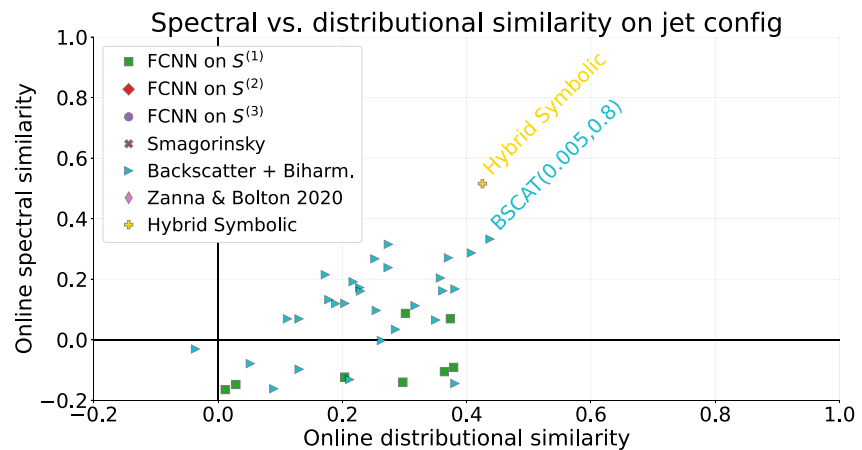


Figure D9. Like Figure 8, but evaluated on jet configuration. In this regime, the only models which appear on the Pareto frontier (highlighted in text) are the hybrid symbolic model and one parameter setting of the backscatter parameterization, which differs significantly from the eddy-configuration Pareto-optimal settings shown in Figures 8 and D8.

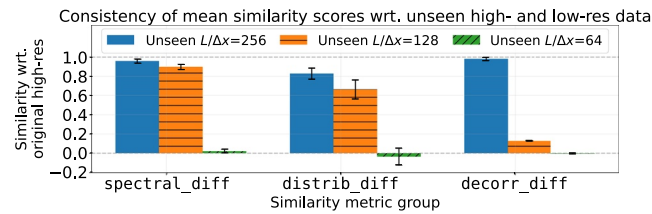


Figure D10. Mean similarity scores for *unseen* high-resolution ($L/\Delta x = 256$), low-resolution ($L/\Delta x = 64$), and intermediate-resolution ($L/\Delta x = 128$) simulations with respect to the actual high- and low-resolution datasets used to evaluate parameterizations. Error-bars show means and standard deviations over 10 random samples of five simulations from a set of 25 unseen simulations. Spectral and decorrelation time similarity scores between different randomly re-run high-res simulations are >0.95 on average (and ≤ 0.01 on average for unseen low-resolution simulations), indicating they are fairly reliable (they should be near 1 for high-res and near 0 for low-res). End-of-simulation distributional similarity scores are a bit noisier, averaging 0.83 for unseen high-resolution simulations (so such scores in our results of above ≈ 0.8 are potentially near-optimal). Although distributional similarity scores are still precise enough to provide meaningful insight into parameterization performance, future experiments could improve their precision by increasing the size of ensembles, or by comparing distributions marginalized over more than just the final timestep. Finally, $L/\Delta x = 128$ simulations score highly (closer to $L/\Delta x = 256$) on distributional and spectral similarity, indicating convergence on long-term “climate” predictions. However, they score much worse (closer to $L/\Delta x = 64$) on decorrelation time similarity, suggesting that short-term “weather” predictions are more sensitive to changes in resolution.

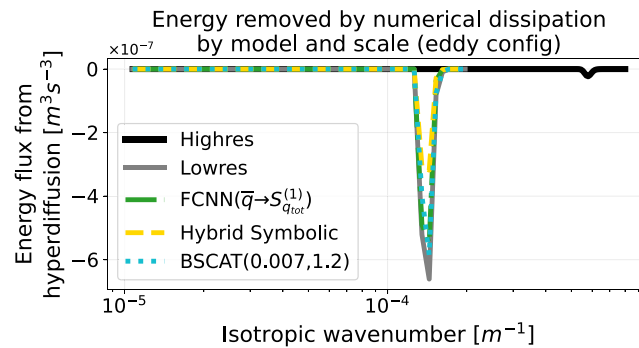


Figure D11. Comparison of the energy density removed via numerical dissipation at each scale for different parameterizations and resolutions (on eddy configuration, with spectra averaged over five simulations). Although the definition of the dissipative term is identical at each resolution, the actual amount of energy dissipated varies in practice due to how parameterizations change the distribution of quantities across scales. In this case, parameterized models lose less energy to numerical dissipation than unparameterized models at the same resolution, likely because the purpose of those parameterizations is to transport energy to larger scales.

KDE-estimated probability density functions (PDFs) of simulation quantities in eddy configuration

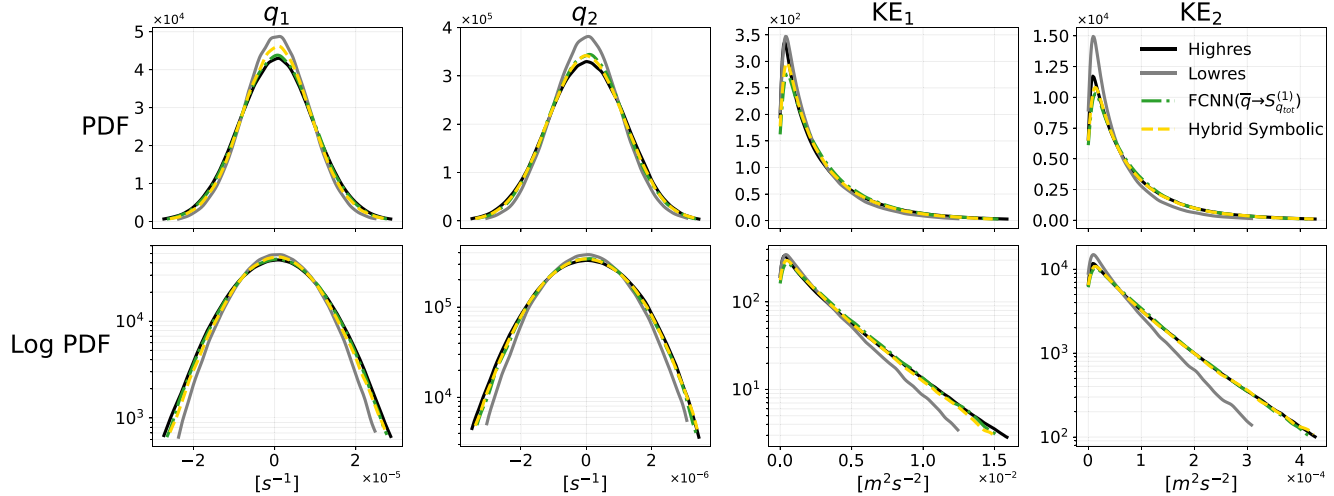


Figure D12. Probability density functions (PDFs), calculated using kernel density estimates (KDE), in both real (top) and logarithmic (bottom) space of upper and lower PV and KE for selected parameterizations (as compared to unparameterized baselines, and computed via kernel density estimation). The selected parameterizations cause these quantities to match the high-resolution simulation much more closely, even in the tails of the distribution (e.g., far right sides of log PDF plots).

Table D1

Wall Clock Times to Train Parameterizations (Center) and Run Simulations (Right) for Different Simulation Types; That Is, for Single Runs on a Tesla V100 for GPUs and an M1 MacBook Pro for CPUs

Simulation Type	Train Time	Runtime
Neural Network (CPU)	—	2 hr 53 min
Neural Network (GPU)	52 min	13 min 4 s
High-res	N/A	5 min 57 s
Hybrid Symbolic	35 min	2 min 22 s
Backscatter + Biharmonic	N/A	59 s
Low-res	N/A	22 s

Note. The FCNN slows down the low-res simulations (Zanna & Bolton, 2020), even when utilizing a GPU. The low-res simulations with FCNN are twice as slow as the high-res; the slow down is primarily due to the depth of the neural network. The symbolic regression-parameterized simulations are more than twice as fast as the high-res simulations. Lower-order backscatter parameterizations are >2x as fast again (though still < $\frac{1}{2}$ x the speed of unparameterized low-res simulations). This is consistent with Zanna and Bolton (2020).

Table D2

“Leaderboard” of Models With the Highest Arithmetic Means (Σ) and Geometric Means (Π) Over Our Three Score Categories (Average Distributional, Spectral, and Decorrelation Time Similarity; That Is, the Axes of the Pareto Frontier Plots in Figure 8, D8, and D9) in Both Eddy and Jet Configuration

Model	Eddy configuration		Jet configuration	
	Rank by Σ	Rank by Π	Rank by Σ	Rank by Π
FCNN($\bar{q} \rightarrow S_{q_{tot}}^{(1)}$)	1	—	129	—
BSCAT(0.02,1.0)	28	1	31	—
Hybrid Symbolic	4	11	1	—
BSCAT(0.005,0.8)	74	23	2	1

Note. Off-diagonal elements show each model’s ranking by score reductions where it is not optimal, and “—” indicates that a negative similarity score was found (so the geometric mean is not meaningful). The hybrid symbolic model ranks highly by all score reductions except its jet configuration geometric mean (where its decorrelation time similarity is slightly negative).

Data Availability Statement

Version 1.0.2 of the Python repository used for training and evaluating parameterizations is preserved at <https://doi.org/10.5281/zenodo.7222704>, available via the MIT license and developed openly at https://github.com/m2lines/pyqg_parameterization_benchmarks (Ross et al., 2022). The baseline high- and low-resolution datasets used for evaluating parameterizations, as well as the subgrid forcing data sets used for training them, are available at Zenodo via <https://doi.org/10.5281/zenodo.6609034> under a Creative Commons Attribution 4.0 International license (Ross, 2022).

Acknowledgments

This research is supported by the generosity of Eric and Wendy Schmidt by recommendation of Schmidt Futures, as part of its Virtual Earth System Research Institute (VESRI). C.F.G. was partially supported by the NSF DMS Grant 2009752. This research was also supported in part through the NYU IT High Performance Computing resources, services, and staff expertise. The authors would like to thank Elizabeth Yankovsky, Ryan Abernathey, Adam Subel, and Tom Beucler for helpful conversations and suggestions; and three reviewers for their comments which helped improved the manuscript.

References

- Abernathey, R., Rocha, C. B., Ross, A., Jansen, M., Li, Z., Poulin, F. J., et al. (2022). pyqg/pyqg: V0.7.2 [Dataset]. Zenodo. <https://doi.org/10.5281/zenodo.6563667>
- Adebayo, J., Gilmer, J., Muelly, M., Goodfellow, I., Hardt, M., & Kim, B. (2018). Sanity checks for saliency maps. *Advances in Neural Information Processing Systems*, 31.
- Anstey, J. A., & Zanna, L. (2017). A deformation-based parametrization of ocean mesoscale eddy Reynolds stresses. *Ocean Modelling*, 112, 99–111. <https://doi.org/10.1016/j.ocemod.2017.02.004>
- Bach, S., Binder, A., Montavon, G., Klauschen, F., Müller, K.-R., & Samek, W. (2015). On pixel-wise explanations for non-linear classifier decisions by layer-wise relevance propagation. *PLoS One*, 10(7), e0130140. <https://doi.org/10.1371/journal.pone.0130140>
- Baehrens, D., Schroeter, T., Harmeling, S., Kawanabe, M., Hansen, K., & Müller, K.-R. (2010). How to explain individual classification decisions. *Journal of Machine Learning Research*, 11, 1803–1831.
- Beck, A., Flad, D., & Munz, C.-D. (2019). Deep neural networks for data-driven LES closure models. *Journal of Computational Physics*, 398, 108910. <https://doi.org/10.1016/j.jcp.2019.108910>
- Beck, A., & Kurz, M. (2021). A perspective on machine learning methods in turbulence modeling. *GAMM-Mitteilungen*, 44(1), e202100002. <https://doi.org/10.1002/gamm.202100002>
- Berloff, P., Ryzhov, E., & Shevchenko, I. (2021). On dynamically unresolved oceanic mesoscale motions. *Journal of Fluid Mechanics*, 920, A41. <https://doi.org/10.1017/jfm.2021.477>
- Berloff, P. S. (2005). Random-forcing model of the mesoscale oceanic eddies. *Journal of Fluid Mechanics*, 529, 71–95. <https://doi.org/10.1017/S0022112005003393>
- Beucler, T., Pritchard, M. S., Yuval, J., Gupta, A., Peng, L., Rasp, S., et al. (2021). Climate-invariant machine learning. CoRR, abs/2112.08440.
- Bolton, T., & Zanna, L. (2019). Applications of deep learning to ocean data inference and subgrid parameterization. *Journal of Advances in Modeling Earth Systems*, 11(1), 376–399. <https://doi.org/10.1029/2018MS001472>
- Brenowitz, N. D., & Bretherton, C. S. (2018). Prognostic validation of a neural network unified physics parameterization. *Geophysical Research Letters*, 45(12), 6289–6298. <https://doi.org/10.1029/2018GL078510>
- Brunton, S. L., Proctor, J. L., & Kutz, J. N. (2016). Sparse identification of nonlinear dynamics with control (SINDYc). *IFAC-PapersOnLine*, 49(18), 710–715. <https://doi.org/10.1016/j.ifacol.2016.10.249>
- Champion, K., Lusch, B., Kutz, J. N., & Brunton, S. L. (2019). Data-driven discovery of coordinates and governing equations. *Proceedings of the National Academy of Sciences*, 116(45), 22445–22451. <https://doi.org/10.1073/pnas.1906995116>
- Cranmer, M. (2020). PySR: Fast & parallelized symbolic regression in Python/Julia [Dataset]. Zenodo. <https://doi.org/10.5281/zenodo.4041459>
- Dresdner, G., Kochkov, D., Norgaard, P., Zepeda-Núñez, L., Smith, J. A., Brenner, M. P., & Hoyer, S. (2022). Learning to correct spectral methods for simulating turbulent flows. arXiv preprint arXiv:2207.00556.
- Fox, D. G., & Orszag, S. A. (1973). Pseudospectral approximation to two-dimensional turbulence. *Journal of Computational Physics*, 11(4), 612–619. [https://doi.org/10.1016/0021-9991\(73\)90141-1](https://doi.org/10.1016/0021-9991(73)90141-1)
- Fox-Kemper, B., Adcroft, A., Böning, C. W., Chassignet, E. P., Curchitser, E., Danabasoglu, G., et al. (2019). Challenges and prospects in ocean circulation models. *Frontiers in Marine Science*, 6. <https://doi.org/10.3389/fmars.2019.00065>
- Fox-Kemper, B., Bachman, S. D., Pearson, B. C., & Reckinger, S. J. (2014). Principles and advances in subgrid modelling for eddy-rich simulations.
- Frezat, H., Sommer, J. L., Fablet, R., Balarac, G., & Lguensat, R. (2022). A posteriori learning for quasi-geostrophic turbulence parametrization. arXiv preprint arXiv:2204.03911.
- Gallet, B., & Ferrari, R. (2021). A quantitative scaling theory for meridional heat transport in planetary atmospheres and oceans. *AGU Advances*, 2(3), e2020AV000362. <https://doi.org/10.1029/2020av000362>
- Gent, P. R., & McWilliams, J. C. (1990). Isopycnal mixing in ocean circulation models. *Journal of Physical Oceanography*, 20(1), 150–155. [https://doi.org/10.1175/1520-0485\(1990\)020<0150:IMOCM>2.0.CO;2](https://doi.org/10.1175/1520-0485(1990)020<0150:IMOCM>2.0.CO;2)
- Gent, P. R., Willebrand, J., McDougall, T. J., & McWilliams, J. C. (1995). Parameterizing eddy-induced tracer transports in ocean circulation models. *Journal of Physical Oceanography*, 25(4), 463–474. [https://doi.org/10.1175/1520-0485\(1995\)025<0463:PEITTI>2.0.CO;2](https://doi.org/10.1175/1520-0485(1995)025<0463:PEITTI>2.0.CO;2)
- Griffies, S., Adcroft, A., Hewitt, H., Oning, C., Chassignet, E., Danabasoglu, G., et al. (2009). Problems and prospects in large-scale ocean circulation models. *OceanObs09 Proceedings*.
- Griffies, S., Winton, M., Anderson, W. G., Benson, R., Delworth, T. L., Dufour, C. O., et al. (2015). Impacts on ocean heat from transient mesoscale eddies in a hierarchy of climate models. *Journal of Climate*, 28(3), 952–977. <https://doi.org/10.1175/JCLI-D-14-00353.1>
- Grooms, I., Loose, N., Abernathey, R., Steinberg, J., Bachman, S. D., Marques, G., et al. (2021). Diffusion-based smoothers for spatial filtering of gridded geophysical data. *Journal of Advances in Modeling Earth Systems*, 13(9), e2021MS002552. <https://doi.org/10.1029/2021ms002552>
- Grooms, I., Nadeau, L.-P., & Smith, K. S. (2013). Mesoscale eddy energy locality in an idealized ocean model. *Journal of Physical Oceanography*, 43(9), 1911–1923. <https://doi.org/10.1175/jpo-d-13-036.1>
- Guan, Y., Chattopadhyay, A., Subel, A., & Hassanzadeh, P. (2022). Stable a posteriori LES of 2D turbulence using convolutional neural networks: Backscattering analysis and generalization to higher re via transfer learning. *Journal of Computational Physics*, 458, 111090. <https://doi.org/10.1016/j.jcp.2022.111090>
- Guillaumin, A. P., & Zanna, L. (2021). Stochastic-deep learning parameterization of ocean momentum forcing. *Journal of Advances in Modeling Earth Systems*, 13(9), e2021MS002534. <https://doi.org/10.1029/2021MS002534>

- Hallberg, R. (2013). Using a resolution function to regulate parameterizations of oceanic mesoscale eddy effects. *Ocean Modelling*, 72, 92–103. <https://doi.org/10.1016/j.ocemod.2013.08.007>
- Hastie, T., Tibshirani, R., & Wainwright, M. (2015). Statistical learning with sparsity. *Monographs on Statistics and Applied Probability*, 143, 143.
- Hewitt, H. T., Roberts, M., Mathiot, P., Biastoch, A., Blockley, E., Chassignet, E. P., et al. (2020). Resolving and parameterising the ocean mesoscale in Earth system models. *Current Climate Change Reports*, 6(4), 137–152. <https://doi.org/10.1007/s40641-020-00164-w>
- Hornik, K., Stinchcombe, M., & White, H. (1989). Multilayer feedforward networks are universal approximators. *Neural Networks*, 2(5), 359–366. [https://doi.org/10.1016/0893-6080\(89\)90020-8](https://doi.org/10.1016/0893-6080(89)90020-8)
- Jansen, M. F., & Held, I. M. (2014). Parameterizing subgrid-scale eddy effects using energetically consistent backscatter. *Ocean Modelling*, 80, 36–48. <https://doi.org/10.1016/j.ocemod.2014.06.002>
- Jansen, M. F., Held, I. M., Adcroft, A., & Hallberg, R. (2015). Energy budget-based backscatter in an eddy permitting primitive equation model. *Ocean Modelling*, 94, 15–26. <https://doi.org/10.1016/j.ocemod.2015.07.015>
- Kent, J., Jablonowski, C., Thuburn, J., & Wood, N. (2016). An energy-conserving restoration scheme for the shallow-water equations. *Quarterly Journal of the Royal Meteorological Society*, 142(695), 1100–1110. <https://doi.org/10.1002/qj.2713>
- Khani, S., & Porté-Agel, F. (2017). Evaluation of non-eddy viscosity subgrid-scale models in stratified turbulence using direct numerical simulations. *European Journal of Mechanics - B: Fluids*, 65, 168–178. <https://doi.org/10.1016/j.euromechflu.2017.03.009>
- Killworth, P. D. (1997). On the parameterization of eddy transfer part i. theory. *Journal of Marine Research*, 55(6), 1171–1197. <https://doi.org/10.1357/002240973224102>
- Kindermans, P.-J., Hooker, S., Adebayo, J., Alber, M., Schütt, K. T., Dähne, S., et al. (2019). The (un) reliability of saliency methods. In *Explainable AI: Interpreting, explaining and visualizing deep learning* (pp. 267–280). Springer.
- Kochkov, D., Smith, J. A., Alieva, A., Wang, Q., Brenner, M. P., & Hoyer, S. (2021). Machine learning-accelerated computational fluid dynamics. *Proceedings of the National Academy of Sciences*, 118(21), e2101784118. <https://doi.org/10.1073/pnas.2101784118>
- Koza, J. R. (1994). Genetic programming as a means for programming computers by natural selection. *Statistics and Computing*, 4(2), 87–112. <https://doi.org/10.1007/bf00175355>
- Kraichnan, R. H. (1976). Eddy viscosity in two and three dimensions. *Journal of the Atmospheric Sciences*, 33(8), 1521–1536. [https://doi.org/10.1175/1520-0469\(1976\)033<1521:evitat>2.0.co;2](https://doi.org/10.1175/1520-0469(1976)033<1521:evitat>2.0.co;2)
- Krasnopolsky, V. M., Fox-Rabinovitz, M. S., Hou, Y. T., Lord, S. J., & Belochitski, A. A. (2010). Accurate and fast neural network emulations of model radiation for the NCEP coupled climate forecast system: Climate simulations and seasonal predictions. *Monthly Weather Review*, 138(5), 1822–1842. <https://doi.org/10.1175/2009MWR3149.1>
- Kullback, S., & Leibler, R. A. (1951). On information and sufficiency. *The Annals of Mathematical Statistics*, 22(1), 79–86. <https://doi.org/10.1214/aoms/1177729694>
- Layton, W. J., & Rebholz, L. G. (2012). *Approximate deconvolution models of turbulence: Analysis, phenomenology and numerical analysis* (Vol. 2042). Springer Science & Business Media.
- Li, H., Zhao, Y., Wang, J., & Sandberg, R. D. (2021). Data-driven model development for large-eddy simulation of turbulence using gene-expression programming. *Physics of Fluids*, 33(12), 125127. <https://doi.org/10.1063/5.0076693>
- Loose, N., Abernathy, R., Grooms, I., Busecke, J., Guillaumin, A., Yankovsky, E., et al. (2022). GCM-filters: A python package for diffusion-based spatial filtering of gridded data. *Journal of Open Source Software*, 7(70), 3947. <https://doi.org/10.21105/joss.03947>
- Lund, T. (1997). On the use of discrete filters for large eddy simulation. *Annual Research Briefs*, 83–95.
- Marques, G. M., Loose, N., Yankovsky, E., Steinberg, J. M., Chang, C.-Y., Bhamidipati, N., et al. (2022). NeverWorld2: An idealized model hierarchy to investigate ocean mesoscale eddies across resolutions. *Geoscientific Model Development*, 15(17), 6567–6579. <https://doi.org/10.5194/gmd-15-6567-2022>
- Marshall, D. P., & Adcroft, A. J. (2010). Parameterization of ocean eddies: Potential vorticity mixing, energetics and Arnold's first stability theorem. *Ocean Modelling*, 32(3–4), 188–204. <https://doi.org/10.1016/j.ocemod.2010.02.001>
- Maulik, R., San, O., Rasheed, A., & Vedula, P. (2019). Subgrid modelling for two-dimensional turbulence using neural networks. *Journal of Fluid Mechanics*, 858, 122–144. <https://doi.org/10.1017/jfm.2018.770>
- Meneveau, C., & Katz, J. (2000). Scale-invariance and turbulence models for large-eddy simulation. *Annual Review of Fluid Mechanics*, 32(1), 1–32. <https://doi.org/10.1146/annurev.fluid.32.1.1>
- Mojgani, R., Chattopadhyay, A., & Hassanzadeh, P. (2021). Closed-form discovery of structural errors in models of chaotic systems by integrating Bayesian sparse regression and data assimilation. arXiv preprint arXiv:2110.00546.
- Monge, G. (1781). Mémoire sur la théorie des déblais et des remblais. *Mémoires de Mathématique et de Physique, présentés à l'Académie Royale des Sciences* (pp. 666–704).
- Natale, A., & Cotter, C. J. (2017). Scale-selective dissipation in energy-conserving finite-element schemes for two-dimensional turbulence. *Quarterly Journal of the Royal Meteorological Society*, 143(705), 1734–1745. <https://doi.org/10.1002/qj.3063>
- O'Gorman, P. A., & Dwyer, J. G. (2018). Using machine learning to parameterize moist convection: Potential for modeling of climate, climate change, and extreme events. *Journal of Advances in Modeling Earth Systems*, 10(10), 2548–2563. <https://doi.org/10.1029/2018MS001351>
- Orszag, S. A. (1971). On the elimination of aliasing in finite-difference schemes by filtering high-wavenumber components. *Journal of the Atmospheric Sciences*, 28(6), 1074–1074. [https://doi.org/10.1175/1520-0469\(1971\)028<1074:oteoai>2.0.co;2](https://doi.org/10.1175/1520-0469(1971)028<1074:oteoai>2.0.co;2)
- Pawar, S., San, O., Rasheed, A., & Vedula, P. (2020). A priori analysis on deep learning of subgrid-scale parameterizations for Kraichnan turbulence. *Theoretical and Computational Fluid Dynamics*, 34(4), 429–455. <https://doi.org/10.1007/s00162-019-00512-z>
- Piomelli, U., Moin, P., & Ferziger, J. H. (1988). Model consistency in large eddy simulation of turbulent channel flows. *The Physics of Fluids*, 31(7), 1884–1891. <https://doi.org/10.1063/1.866635>
- Pope, S. B. (2000). *Turbulent flows*. Cambridge University Press.
- Porta Mana, P., & Zanna, L. (2014). Toward a stochastic parameterization of ocean mesoscale eddies. *Ocean Modelling*, 79, 1–20. <https://doi.org/10.1016/j.ocemod.2014.04.002>
- Rasp, S., Pritchard, M. S., & Gentile, P. (2018). Deep learning to represent subgrid processes in climate models. *Proceedings of the National Academy of Sciences*, 115(39), 9684–9689. <https://doi.org/10.1073/pnas.1810286115>
- Recht, B., Roelofs, R., Schmidt, L., & Shankar, V. (2018). Do cifar-10 classifiers generalize to cifar-10? arXiv preprint arXiv:1806.00451.
- Redi, M. H. (1982). Oceanic isopycnal mixing by coordinate rotation. *Journal of Physical Oceanography*, 12(10), 1154–1158. [https://doi.org/10.1175/1520-0485\(1982\)012<1154:oimbc>2.0.co;2](https://doi.org/10.1175/1520-0485(1982)012<1154:oimbc>2.0.co;2)
- Ronneberger, O., Fischer, P., & Brox, T. (2015). U-net: Convolutional networks for biomedical image segmentation. In *International conference on medical image computing and computer-assisted intervention* (pp. 234–241).
- Rosenblatt, M. (1956). Remarks on some nonparametric estimates of a density function. *The Annals of Mathematical Statistics*, 27(3), 832–837. <https://doi.org/10.1214/aoms/1177728190>

- Ross, A. S. (2022). pyqg subgrid forcing datasets [Dataset]. Zenodo. <https://doi.org/10.5281/zenodo.6609035>
- Ross, A. S., Zanna, L., & Perezhagin, P. (2022). m2lines/pyqg_parameterization_benchmarks: V1.0.2 [Software]. Zenodo. <https://doi.org/10.5281/zenodo.7222704>
- Rubner, Y., Tomasi, C., & Guibas, L. J. (2000). The Earth Mover's Distance as a metric for image retrieval. *International Journal of Computer Vision*, 40(2), 99–121. <https://doi.org/10.1023/a:1026543900054>
- Rudy, S. H., Brunton, S. L., Proctor, J. L., & Kutz, J. N. (2017). Data-driven discovery of partial differential equations. *Science Advances*, 3(4), e1602614. <https://doi.org/10.1126/sciadv.1602614>
- Sagaut, P. (2006). *Large eddy simulation for incompressible flows: An introduction*. Springer Science & Business Media.
- Sagaut, P., & Grohens, R. (1999). Discrete filters for large eddy simulation. *International Journal for Numerical Methods in Fluids*, 31(8), 1195–1220. [https://doi.org/10.1002/\(sici\)1097-0363\(19991230\)31:8<1195::aid-fld914>3.0.co;2-h](https://doi.org/10.1002/(sici)1097-0363(19991230)31:8<1195::aid-fld914>3.0.co;2-h)
- Salmon, R. (1980). Baroclinic instability and geostrophic turbulence. *Geophysical & Astrophysical Fluid Dynamics*, 15(1), 167–211. <https://doi.org/10.1080/03091928008241178>
- Schmidt, M., & Lipson, H. (2009). Distilling free-form natural laws from experimental data. *Science*, 324(5923), 81–85. <https://doi.org/10.1126/science.1165893>
- Schneider, T., Teixeira, J., Bretherton, C. S., Brient, F., Pressel, K. G., Schär, C., & Siebesma, A. P. (2017). Climate goals and computing the future of clouds. *Nature Climate Change*, 7(1), 3–5. <https://doi.org/10.1038/nclimate3190>
- Shamekh, S., Lamb, K. D., Huang, Y., & Gentile, P. (2022). Implicit learning of convective organization explains precipitation stochasticity. *Earth and Space Science Open Archive*, 16. <https://doi.org/10.1002/essoar.10512517.1>
- Shevchenko, I., & Berloff, P. (2021). On a minimum set of equations for parameterisations in comprehensive ocean circulation models. *Ocean Modelling*, 168, 101913. <https://doi.org/10.1016/j.ocemod.2021.101913>
- Sirignano, J., MacArt, J. F., & Freund, J. B. (2020). DPM: A deep learning PDE augmentation method with application to large-eddy simulation. *Journal of Computational Physics*, 423, 109811. <https://doi.org/10.1016/j.jcp.2020.109811>
- Smagorinsky, J. (1963). General circulation experiments with the primitive equations: I. The basic experiment. *Monthly Weather Review*, 91(3), 99–164. [https://doi.org/10.1175/1520-0493\(1963\)091<0099:gcwptp>2.3.co;2](https://doi.org/10.1175/1520-0493(1963)091<0099:gcwptp>2.3.co;2)
- Springenberg, J. T., Dosovitskiy, A., Brox, T., & Riedmiller, M. (2014). Striving for simplicity: The all convolutional net. arXiv preprint arXiv:1412.6806.
- Stephens, T. (2019). Genetic Programming in Python, with a scikit-learn inspired API. Retrieved from <https://gplearn.readthedocs.io/en/stable>
- Stevens, B., & Bony, S. (2013). What are climate models missing? *Science*, 340(6136), 1053–1054. <https://doi.org/10.1126/science.1237554>
- Stoffer, R., Van Leeuwen, C. M., Podareanu, D., Codreanu, V., Veerman, M. A., Janssens, M., et al. (2021). Development of a large-eddy simulation subgrid model based on artificial neural networks: A case study of turbulent channel flow. *Geoscientific Model Development*, 14(6), 3769–3788. <https://doi.org/10.5194/gmd-14-3769-2021>
- Subel, A., Chattopadhyay, A., Guan, Y., & Hassanzadeh, P. (2021). Data-driven subgrid-scale modeling of forced burgers turbulence using deep learning with generalization to higher Reynolds numbers via transfer learning. *Physics of Fluids*, 33(3), 031702. <https://doi.org/10.1063/5.0040286>
- Subel, A., Guan, Y., Chattopadhyay, A., & Hassanzadeh, P. (2022). Explaining the physics of transfer learning a data-driven subgrid-scale closure to a different turbulent flow. arXiv preprint arXiv:2206.03198.
- Thuburn, J., Kent, J., & Wood, N. (2014). Cascades, backscatter and conservation in numerical models of two-dimensional turbulence. *Quarterly Journal of the Royal Meteorological Society*, 140(679), 626–638. <https://doi.org/10.1002/qj.2166>
- Turing, A. (1950). I—Computing machinery and intelligence. *Mind*, 59(236), 433–460. <https://doi.org/10.1093/mind/LIX.236.433>
- Um, K., Brand, R., Fei, Y. R., Holl, P., & Thuerey, N. (2020). Solver-in-the-loop: Learning from differentiable physics to interact with iterative PDE-solvers. *Advances in Neural Information Processing Systems*, 33, 6111–6122.
- Vallis, G. K. (2017). *Atmospheric and oceanic fluid dynamics*. Cambridge University Press.
- Vallis, G. K., & Hua, B.-I. (1988). Eddy viscosity of the anticipated potential vorticity method. *Journal of the Atmospheric Sciences*, 45(4), 617–627. [https://doi.org/10.1175/1520-0469\(1988\)045<0617:evotap>2.0.co;2](https://doi.org/10.1175/1520-0469(1988)045<0617:evotap>2.0.co;2)
- Xie, C., Wang, J., & Weinan, E. (2020). Modeling subgrid-scale forces by spatial artificial neural networks in large eddy simulation of turbulence. *Physical Review Fluids*, 5(5), 054606. <https://doi.org/10.1103/physrevfluids.5.054606>
- Xing, H., Zhang, J., Ma, W., & Wen, D. (2022). Using gene expression programming to discover macroscopic governing equations hidden in the data of molecular simulations. *Physics of Fluids*, 34(5), 057109. <https://doi.org/10.1063/5.0090134>
- Yankovsky, E., Zanna, L., & Smith, K. S. (2022). Influences of mesoscale ocean eddies on flow vertical structure in a resolution-based model hierarchy. *Journal of Advances in Modeling Earth Systems*, 14(11), e2022MS003203. <https://doi.org/10.1029/2022ms003203>
- Yuval, J., & O’Gorman, P. A. (2021). Neural-network parameterization of subgrid momentum transport in the atmosphere. *Earth and Space Science Open Archive*, 15. <https://doi.org/10.1002/essoar.10507557.1>
- Yuval, J., O’Gorman, P. A., & Hill, C. N. (2021). Use of neural networks for stable, accurate and physically consistent parameterization of subgrid atmospheric processes with good performance at reduced precision. *Geophysical Research Letters*, 48(6), e2020GL091363. <https://doi.org/10.1029/2020GL091363>
- Zanna, L., Bachman, S., & Jansen, M. (2020). Energizing turbulence closures in ocean models. *CLIVAR Exchanges/US CLIVAR Variations*, 18(1), 3–8. <https://doi.org/10.5065/g8w0-fy32>
- Zanna, L., & Bolton, T. (2020). Data-driven equation discovery of ocean mesoscale closures. *Geophysical Research Letters*, 47(17), e2020GL088376. <https://doi.org/10.1029/2020GL088376>
- Zanna, L., & Bolton, T. (2021). Deep learning of unresolved turbulent ocean processes in climate models. In *Deep learning for the Earth sciences* (pp. 298–306). John Wiley & Sons, Ltd. <https://doi.org/10.1002/9781119646181.ch20>
- Zhang, S., & Lin, G. (2018). Robust data-driven discovery of governing physical laws with error bars. *Proceedings of the Royal Society A: Mathematical, Physical & Engineering Sciences*, 474(2217), 20180305. <https://doi.org/10.1098/rspa.2018.0305>
- Zhou, Z., He, G., Wang, S., & Jin, G. (2019). Subgrid-scale model for large-eddy simulation of isotropic turbulent flows using an artificial neural network. *Computers & Fluids*, 195, 104319. <https://doi.org/10.1016/j.compfluid.2019.104319>